

Rapport réalisé par

Matthieu FERRETE - Louis VIRY
Tom CHAOUACHI - Théo MAUNY
Mathilde CARIOU - Alana DONNAT

Département Électronique
et Télécommunications

4^{ème} année 2025 - 2026

Réalisation d'un robot ramasseur de balles de Tennis : «R2B2»

Enseignements
DET07-CDC, DET08-PRJ

Enseignants
Christophe Lemoine
François Yven

Remerciements

Nous tenons à remercier les enseignants qui encadrent ce projet pour leur accompagnement tout au long de cette année.

Nous remercions M. Christophe LEMOINE pour son encadrement pédagogique et M. François YVEN pour son suivi en tant que tuteur de projet.

Nous tenons à adresser nos remerciements les plus sincères à M. Gilles PICOULT, dont la présence a été déterminante dans la progression de ce projet. Ses conseils techniques avisés, sa disponibilité, les solutions qu'il nous a proposées face aux difficultés rencontrées, la fabrication des PCB, ainsi que ses encouragements ont été d'une valeur inestimable tout au long de la réalisation de R2B2.

R2B2

Robot Ramasseur de Balles de Tennis

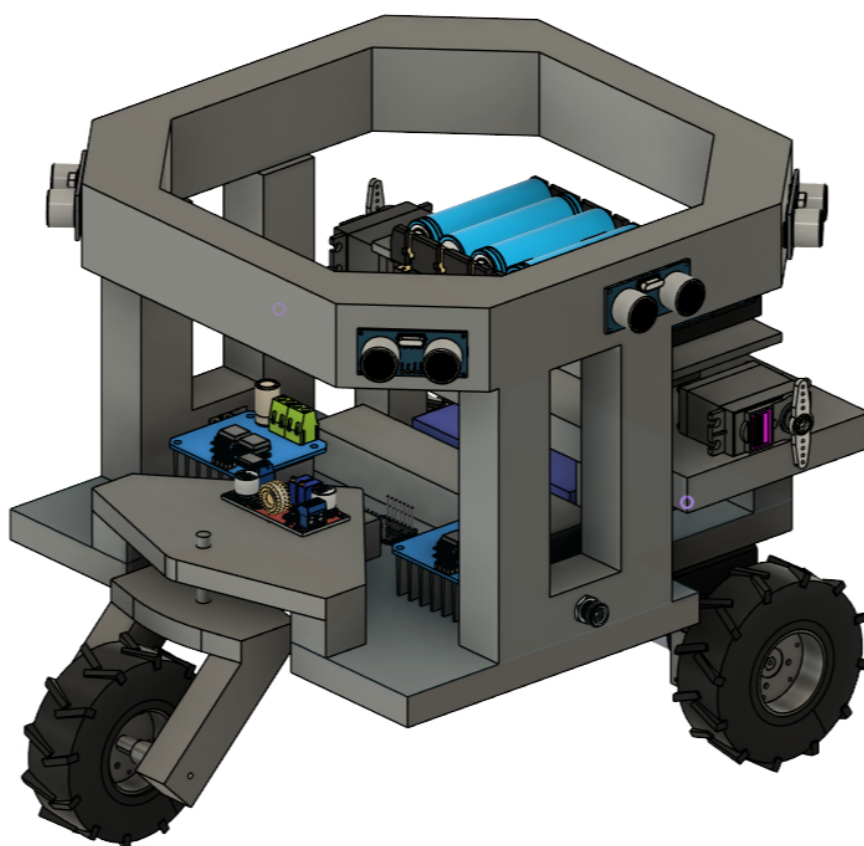


Table des matières

1	Introduction	7
2	Présentation du projet	7
2.1	Analyse du besoin	8
2.1.1	Optimisation du temps de jeu	8
2.1.2	Sécurité et prévention des blessures	8
2.1.3	Accessibilité et Handisport	9
2.2	État de l’art	9
2.2.1	Les solutions manuelles	9
2.2.2	Les solutions automatisées	10
2.2.3	Positionnement de R2B2	11
2.3	Solution retenue	12
2.3.1	Sécurité et détection	12
2.3.2	Compacité et discrétion	12
2.3.3	Performances et autonomie	12
2.3.4	Ramassage	13
3	Cahier des charges	13
3.1	Contraintes fonctionnelles	14
3.2	Contraintes techniques	14
3.2.1	Niveaux logiques et alimentation	14
3.2.2	Architecture de contrôle hybride	15
3.2.3	Protocoles de communication	15
3.3	Ajustements stratégiques en cours de projet	15
3.3.1	Fonctionnalités abandonnées en début de projet	15
3.3.2	Remplacement du microcontrôleur principal	16
4	Architecture générale du système	16
4.1	Schéma de principe général	17
4.2	Architecture de contrôle hybride FPGA + Microcontrôleur	18
4.2.1	Chaîne de perception visuelle	18
4.2.2	Couche décisionnelle et temps réel	18
4.2.3	Chaîne d’actionnement	18
4.3	Outils et environnements de développement	19
4.3.1	FPGA — Intel Quartus Prime	19
4.3.2	Microcontrôleurs — Arduino IDE	19
4.3.3	Supervision PC — Python	19
4.3.4	Conception mécanique — Autodesk Fusion 360	19

5	Module de déplacement et motorisation	19
5.1	Choix et dimensionnement des actionneurs	20
5.1.1	Expression du besoin mécanique	20
5.1.2	Sélection et validation : JGB37-520 24 V 960 RPM	21
5.2	Pièces imprimées dédiées aux actionneurs	21
5.2.1	Adaptateur roue-moteur	21
5.2.2	Couronne magnétique sur l'arbre moteur	21
5.2.3	Support SS49E	22
5.3	Mesure de vitesse par capteur à effet Hall SS49E	23
5.3.1	Principe de mesure et choix du capteur	23
5.3.2	Première approche : circuit de conditionnement analogique (non conservé)	23
5.3.3	Solution retenue : acquisition ADC sur FPGA en VHDL	24
5.4	Contrôleur moteur BTS en VHDL	26
5.4.1	PWM	26
5.4.2	Arrêt d'urgence	26
5.5	Asservissement en vitesse : régulateur PI en VHDL	26
5.5.1	Architecture : deux régulateurs PI indépendants	26
5.5.2	Limitation et anti-windup	27
5.5.3	État d'avancement	27
5.6	Stratégie de déplacement et navigation	27
5.6.1	Stratégies étudiées	27
5.6.2	Stratégie retenue : parcours en O puis en 8	27
5.6.3	Implémentation envisagée	28
6	Module de détection et navigation	28
6.1	Détection d'obstacles par ultrasons	29
6.1.1	Conversion temps $\rightarrow$ distance	29
6.1.2	Adaptation électrique : pont diviseur de tension	30
6.1.3	Placement sur le robot	30
6.2	Détection des balles par modèle d'apprentissage	31
6.2.1	Matériel : XIAO ESP32-S3 Sense	31
6.2.2	Première approche : filtre de couleur HSV (abandonnée)	31
6.2.3	Modèle d'apprentissage sur Edge Impulse	32
6.2.4	Traitement des données et sélection de la cible	33
6.2.5	Estimation de la distance et de l'angle	33
6.2.6	Transmission des coordonnées vers la DE10-Lite	34
7	Conception mécanique et CAO	34
7.1	Structure et châssis	35
7.1.1	Conception et choix des matériaux	35
7.1.2	Architecture multi-niveaux	36
7.1.3	Répartition des masses et ergonomie	36
7.2	Bras de ramassage : cinématique et actionnement	36
7.2.1	Problème cinématique	36
7.2.2	Courbes paramétriques	37

7.2.3	Actionnement	37
7.2.4	État d'avancement	38
7.3	Pince de saisie	39
7.3.1	Solutions étudiées et choix	39
7.3.2	Principe mécanique — transmission par vis-écrou	39
7.3.3	Actionnement — 28BYJ-48 et driver ULN2003	40
7.3.4	État d'avancement	40
7.4	Roue folle	40
7.4.1	Conception initiale et problème identifié	40
7.4.2	Solution retenue — axe déporté	40
8	Système de batteries et gestion d'énergie	40
8.1	Pack batterie et régulation	41
8.2	Circuit de détection de seuils	41
9	Supervision et interface opérateur	42
9.1	Transmission UART — Trame structurée	43
9.2	Multiplexage UART — Deux canaux ADC	43
9.3	Visualisation série — Python Matplotlib	44
9.4	Interface de supervision — Tennis Robot Monitor	45
9.5	État d'avancement	46
10	Gestion	46
10.1	Gestion du temps — Diagramme de Gantt	47
10.2	Gestion financière	49
11	Démonstration	50
11.1	Scénario de démonstration	51
11.1.1	Mesure de vitesse par capteur à effet Hall et transmission UART	51
11.1.2	Système de préhension — ouverture et fermeture de la pince	51
11.1.3	Détection d'obstacles par capteur à ultrasons	51
11.1.4	Interface de supervision graphique	52
11.1.5	Détection de balle par apprentissage	52
11.2	Difficultés rencontrées et solutions apportées	52
11.2.1	Circuit analogique SS49E — Signal inexploitable	52
11.2.2	Roue folle — Axe recentré a posteriori	52
11.2.3	FSM UART — Données instables à l'envoi	53
11.2.4	Pack batterie — Incident thermique et mesures correctives	53
11.3	Travaux restants pour finaliser le robot	53
11.3.1	Boucle de contrôle vitesse — Double PI synchronisé	53
11.3.2	Réception UART FPGA ← ESP32-C3	54
11.3.3	Algorithme de déplacement — ESP32-C3	54
11.3.4	Traitement des coordonnées balles — FPGA	54
11.3.5	Intégration des capteurs ultrasons	54
11.3.6	Lien Bluetooth et supervision	54
11.3.7	Campagne de tests et intégration système	54

12 Conclusion	54
13 Bibliographie	55

1. Introduction

L'évolution technologique dans le domaine du sport a considérablement transformé la pratique de haut niveau comme celle des amateurs. Si les équipements ont bénéficié d'innovations majeures en science des matériaux, la logistique même du jeu de tennis est restée remarquablement classique : la gestion des balles sur le terrain repose encore presque exclusivement sur l'intervention humaine.

Le tennis est un sport structurellement fragmenté : le temps de jeu effectif ne représente qu'une fraction minime du temps total, déséquilibre encore aggravé lors des entraînements amateurs où le ramassage des balles peut mobiliser jusqu'à 20% à 30% d'une séance. Ces balles immobiles constituent par ailleurs un risque physique avéré, une part significative des blessures survenant non pas à cause de l'effort, mais par un appui instable sur une balle qui traîne.

La dimension inclusive mérite également d'être soulignée : pour les joueurs en fauteuil roulant, le ramassage impose des manœuvres supplémentaires et une flexion du tronc qui peut s'avérer éprouvante, voire impossible selon la pathologie du joueur.

Notre projet, baptisé **R2B2**, vise à concevoir une solution mécatronique autonome opérant **en continu durant le match**. Contrairement aux rares solutions existantes qui nettoient le terrain en fin de session, R2B2 cohabite avec les joueurs, identifie les balles mortes et les collecte en temps réel, sans jamais interférer avec le jeu ni gêner les déplacements des sportifs — ce qui impose des contraintes fortes en détection, navigation, sécurité et réactivité.

Ce document présente la démarche de conception de R2B2 : après une analyse du besoin et un état de l'art, nous décrirons l'architecture générale puis chacun des sous-systèmes — déplacement, détection, ramassage, gestion d'énergie et supervision — avant de conclure par le bilan et les perspectives d'évolution.

2. Présentation du projet

2.1 Analyse du besoin

Pour justifier la pertinence de ce projet, nous avons analysé les impacts du ramassage manuel sous trois angles fondamentaux : l'efficacité temporelle, la sécurité physique et l'inclusion sociale.

2.1.1 Optimisation du temps de jeu

Le tennis souffre d'un ratio temps de jeu sur temps de présence particulièrement faible. Une étude statistique relayée par le *Wall Street Journal* a mis en lumière que sur une rencontre professionnelle d'une durée de trois heures, le temps de jeu effectif ne représente que **17,5% du temps total**, soit à peine 21 minutes d'action réelle [1].

Ce constat est encore plus sévère lors des entraînements amateurs. Lorsqu'un joueur effectue une série de services ou un exercice au panier, le ramassage manuel des 50 à 100 balles dispersées peut prendre entre 5 et 8 minutes. Sur une séance d'une heure, la répétition de cette tâche empiète sur le temps de pratique de **20% à 30%**. L'automatisation de ce processus permettrait de récupérer ce temps perdu pour le consacrer à l'apprentissage technique et au plaisir du jeu.

2.1.2 Sécurité et prévention des blessures

La présence de balles immobiles sur la surface de jeu constitue un risque avéré pour l'intégrité physique des joueurs. Les déplacements au tennis sont caractérisés par des courses latérales, des reprises d'appuis violentes et des reculs précipités pour se replacer en fond de court. Une part significative des accidents survient non pas à cause de l'effort lui-même, mais par un appui instable sur une balle qui traîne.

Si un joueur pose le pied sur une balle lors d'un de ces mouvements, la cheville subit une torsion immédiate. Selon les données de la Fédération Française de Tennis et des études en traumatologie sportive, **l'entorse de la cheville représente environ 25% des blessures recensées au tennis** [2]. En assurant un nettoyage continu de la zone d'évolution, notre robot agit comme un dispositif de sécurité active permettant de réduire fortement la probabilité de chute et de lésion ligamentaire.

2.1.3 Accessibilité et Handisport

L'aspect inclusif est un pilier central de notre réflexion. Pour les joueurs de tennis en fauteuil roulant, la contrainte du ramassage est décuplée. La biomécanique du ramassage en position assise impose des manœuvres supplémentaires avec le fauteuil et une flexion du tronc qui peut s'avérer éprouvante physiquement, voire impossible selon la pathologie du joueur.

Cette difficulté constitue parfois un frein à la pratique ou une source de découragement pour les débutants. En déléguant cette tâche à un système autonome, nous redonnons de l'autonomie aux joueurs en situation de handicap, leur permettant de se focaliser uniquement sur leur performance sportive et leur plaisir de jeu, sans dépendre d'une tierce personne.

2.2 État de l'art

Avant de définir notre solution, nous avons étudié les technologies existantes — manuelles et automatisées — afin d'identifier les lacunes du marché actuel et les bonnes pratiques à conserver.

2.2.1 Les solutions manuelles

- **Le tube de ramassage**

C'est un tube en plastique ou en métal muni d'un clapet anti-retour. Pour faire entrer la balle, on appuie simplement dessus.

Avantage : Très peu coûteux (moins de 20 €), facilement transportable, évite de se baisser complètement.

Inconvénient : Effort physique répétitif, ramassage balle par balle, aucune automatisation possible.

- **Le rouleau de ramassage**

Il s'agit d'un rouleau que l'on pousse sur le court, conçu avec des barreaux souples qui s'écartent lorsqu'on roule sur une balle, permettant de la récupérer et de la stocker dans un panier intégré.

Avantage : Ramassage rapide, grande capacité de stockage (jusqu'à 60 balles).

Inconvénient : Encombrant, nécessite toujours une action humaine continue, inutilisable pendant le jeu.



FIGURE 1 – *Tube de ramassage*



FIGURE 2 – *Rouleau de ramassage*

2.2.2 Les solutions automatisées

- **Le Tennibot**

Robot ramasseur autonome dont le fonctionnement s'inspire de l'aspirateur robot, adapté au tennis. Son système de navigation repose sur l'intelligence artificielle et un LIDAR 360° offrant une portée de 30 mètres [3].

Avantage : Autonomie totale — le joueur joue pendant que le robot ramasse.

Inconvénient : Prix prohibitif pour un usage amateur (2 245 \$), vitesse de ramassage lente, capacité de stockage limitée, opération uniquement hors-match.

- **Le Slinger Bag**

Système robotisé permettant la *distribution* de balles selon plusieurs paramètres : fréquence, vitesse, effets.

Avantage : Sessions d'entraînement personnalisées sans partenaire.

Inconvénient : Conçu exclusivement pour la distribution — aucune fonction de collecte.

L'utilisateur doit interrompre l'entraînement pour ramasser manuellement les balles et recharger la machine.



FIGURE 3 – *Tennibot*



FIGURE 4 – *Slinger Bag*

2.2.3 Positionnement de R2B2

Il apparaît que les systèmes existants présentent des limites structurelles majeures. Les outils manuels imposent une charge physique répétitive et interrompent systématiquement le jeu. Les solutions automatisées comme le Tennibot affichent un coût prohibitif et opèrent uniquement entre les sessions, pas pendant le match.

R2B2 répond à une lacune claire : aucune solution actuelle n'est à la fois abordable financièrement, capable d'opérer en temps réel pendant le jeu, et respectueuse de la sécurité des joueurs. C'est précisément l'objectif que notre projet se donne.

2.3 Solution retenue

Pour répondre aux besoins identifiés, notre projet se fixe quatre objectifs techniques précis qui ont guidé l'ensemble des choix de conception.

2.3.1 Sécurité et détection

La priorité absolue est la cohabitation homme-machine. Le robot doit cartographier son environnement immédiat pour distinguer une balle inerte d'un joueur en mouvement. Si un humain pénètre dans un rayon de **1 mètre** autour du robot, le système doit déclencher un arrêt total ou une manœuvre d'évitement. La chaîne de traitement allant de la détection à l'arrêt des moteurs doit respecter une latence **inférieure à 100 millisecondes**, validée expérimentalement à **85 ms** lors de nos tests.

2.3.2 Compacité et discrétion

Le robot ne doit pas devenir une nuisance visuelle ni perturber les joueurs. Sa **hauteur totale est contenue sous 25 cm**, lui permettant de circuler librement sous le filet et de rester hors du champ de vision direct des joueurs lors des phases de frappe. L'architecture à entraînement différentiel sur trois roues permet des demi-tours instantanés sans avance, indispensables pour s'extraire des coins du court. La motorisation est isolée phoniquement pour rester **sous le seuil des 50 dB**.

2.3.3 Performances et autonomie

Pour être efficace entre deux points, le robot doit être rapide. Une **vitesse de pointe de 3 m/s** est atteinte, permettant d'intervenir rapidement durant les temps morts.

L'alimentation repose sur un pack **7S1P composé de cellules Samsung INR18650-25R**, offrant une tension nominale de **25,9 V** (29,4 V en charge complète, coupure à 17,5 V) pour une capacité de **2 500 mAh**, soit une énergie totale disponible d'environ **64,75 Wh**. Cette configuration garantit une autonomie suffisante pour couvrir la durée moyenne d'un match complet, tout en délivrant un courant de décharge continu pouvant atteindre **20 A** pour alimenter les pics de consommation des moteurs.

2.3.4 Ramassage

Le système de ramassage doit être universel et robuste. Le **taux de réussite de la collecte au premier passage doit dépasser 90%**, même si la balle est encore légèrement en mouvement. Le réservoir embarqué est dimensionné pour contenir entre **10 et 15 balles**, correspondant à la quantité généralement utilisée lors d'un cycle de jeu avant un ramassage complet.

3. Cahier des charges

3.1 Contraintes fonctionnelles

Le tableau suivant synthétise les exigences fonctionnelles retenues pour R2B2, issues de l'analyse du besoin conduite au semestre précédent.

Fonction	Exigence	Justification
Vitesse de déplacement	$\geq 3 \text{ m/s}$, atteinte en $< 4 \text{ s}$	Intervenir efficacement entre deux points
Gabarit	Hauteur $\leq 25 \text{ cm}$	Passer librement sous le filet (hauteur 91 cm au centre)
Autonomie	$\geq 2 \text{ h}$	Couvrir la durée moyenne d'un match complet
Niveau sonore	$\leq 50 \text{ dB}$	Ne pas perturber la concentration des joueurs
Taux de collecte	$\geq 90\%$ au 1 ^{er} passage	Fiabilité opérationnelle en condition réelle
Capacité du réservoir	10 à 15 balles	Correspondance avec un cycle de jeu standard
Budget	$\leq 120 \text{ €}$	Contrainte imposée par le cadre pédagogique

TABLE 1 – Contraintes fonctionnelles de R2B2

3.2 Contraintes techniques

3.2.1 Niveaux logiques et alimentation

L'architecture électronique de R2B2 repose sur une cohabitation de deux domaines de tension. Le **FPGA DE10-Lite** opère en logique **3,3 V**, tandis que les capteurs ultrasons HC-SR04 délivrent leurs signaux Echo en **5 V**. Un pont diviseur résistif ($R_1 = 3,3 \text{ k}\Omega$, $R_2 = 1,8 \text{ k}\Omega$) est donc interposé sur chaque ligne Echo pour adapter les niveaux sans endommager les entrées du FPGA.

L'ensemble du système est alimenté par le pack **7S1P Samsung INR18650-25R** (25,9 V nominal). Des régulateurs de tension assurent la distribution aux différents sous-systèmes selon leurs besoins propres.

3.2.2 Architecture de contrôle hybride

La répartition des tâches entre les deux cartes principales découle d'une contrainte forte : certaines opérations exigent un déterminisme temporel strict que seul un FPGA peut garantir.

- **FPGA DE10-Lite** : gestion temps réel des signaux PWM moteurs, lecture des encodeurs à effet Hall, pilotage des 8 capteurs ultrasons, asservissement PID.
- **XIAO ESP32-C3** : logique décisionnelle de haut niveau — navigation, gestion des trajectoires, réception des coordonnées de balle depuis la caméra, relais des données de supervision vers le PC.
- **XIAO ESP32-S3 Sense** : acquisition et traitement d'image, pilotage du servomoteur SG90, calcul du barycentre et transmission des coordonnées.

3.2.3 Protocoles de communication

Les échanges inter-cartes reposent exclusivement sur l'**UART**, choisi pour sa simplicité d'implémentation en VHDL et sa robustesse. La trame définie entre le FPGA et l'ESP32-C3 est constituée de **8 octets** : un octet de start, un octet d'état (déplacement / bras / pince), un octet de sens de marche, deux octets de consigne de vitesse et deux octets d'angle de braquage.

3.3 Ajustements stratégiques en cours de projet

Le périmètre fonctionnel initial a évolué au fil des réunions d'avancement. Chaque abandon a été motivé par une analyse coût-bénéfice rigoureuse, privilégiant la fiabilité du cœur de fonctionnalité sur l'exhaustivité des options.

3.3.1 Fonctionnalités abandonnées en début de projet

Trois fonctionnalités initialement envisagées ont été retirées dès la première révision du Gantt, le projet ayant été jugé trop ambitieux au regard du calendrier disponible [1] :

- **Détection sonore des balles** : l'idée d'exploiter le son d'impact balle-sol a été écartée en raison de la complexité du traitement du signal en environnement bruyé extérieur (vent, bruits de match), et parce que la vision s'est révélée plus précise et moins énergivore.
- **Renvoi des balles ramassées** : un mécanisme de relance avait été envisagé, mais sa conception mécanique dépassait le budget et le temps disponible sans apporter de valeur déterminante à la démonstration.
- **Base deux roues auto-équilibrée** : le robot était initialement prévu sur deux roues avec un système de contre-balancement. Cette solution a été abandonnée car la correction continue de l'inclinaison complexifiait fortement la détection et le ramassage. Le passage à **trois roues** (deux motrices à l'avant, une roue folle à l'arrière) a permis de conserver l'agilité tout en garantissant la stabilité.

3.3.2 Remplacement du microcontrôleur principal

En cours de développement, le microcontrôleur **XIAO MG24** initialement retenu pour la logique décisionnelle a présenté un dysfonctionnement matériel ne permettant plus son utilisation. Afin de ne pas bloquer l'avancement du projet, il a été remplacé par un **XIAO ESP32-C3** disponible en stock.

Ce choix s'est avéré pertinent : l'ESP32-C3 offre des capacités comparables pour notre usage — communication UART, logique de navigation, gestion des entrées/sorties — sans nécessiter de refonte architecturale. L'ensemble du protocole de communication défini avec le MG24 a été conservé tel quel.

4. Architecture générale du système

4.1 Schéma de principe général

Le robot repose sur une architecture trois niveaux : acquisition sensorielle, traitement décisionnel et actionnement. Deux microprocesseurs et un FPGA se répartissent les tâches selon leur nature — temps réel strict, traitement d'image et logique de navigation — et communiquent par liaisons série UART.

Durant la phase de conception, le port USB-UART natif de l'ESP32-C3 a servi d'interface de débogage principale. Les variables internes — angle de consigne, distance estimée, état du robot — étaient transmises en continu vers un terminal série, permettant de valider le comportement du système sans instrumentation matérielle supplémentaire. Ce canal n'est pas utilisé en conditions opérationnelles.

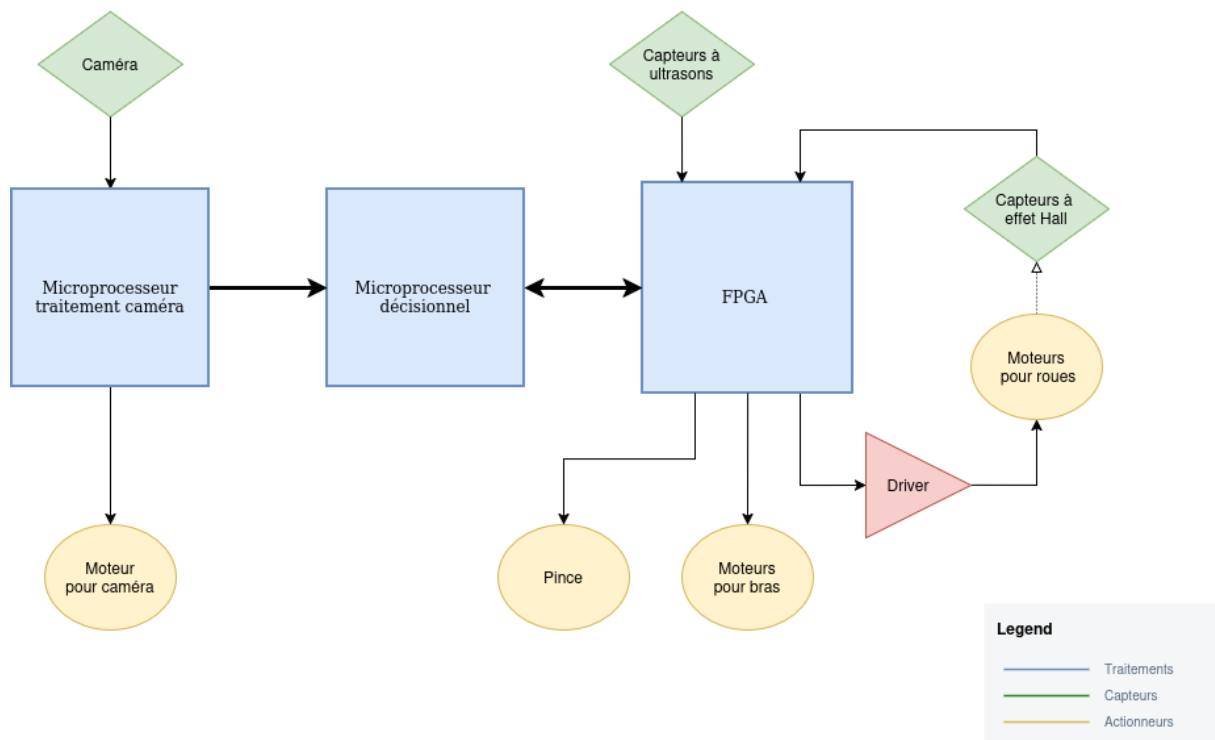


FIGURE 5 – Schéma de principe général du robot ramasseur de balles

4.2 Architecture de contrôle hybride FPGA + Microcontrôleur

La répartition des tâches repose sur les caractéristiques intrinsèques de chaque unité : le **FPGA DE10-Lite** assure la logique câblée déterministe (PID, PWM, capteurs, sécurité) en parallèle à 50 MHz, tandis que le **XIAO ESP32-C3** gère la logique décisionnelle de haut niveau — navigation, séquences de collecte, arbitrage des ordres issus du module caméra.

4.2.1 Chaîne de perception visuelle

Un **microprocesseur dédié au traitement caméra** (XIAO ESP32-S3 Sense) acquiert les images via l'OV3660 et exécute l'inférence FOMO. Il transmet à l'ESP32-C3 l'angle de rotation et le seuil de distance de la balle détectée. Un **moteur orienteur de caméra** permet d'ajuster le champ de vision selon les consignes reçues.

4.2.2 Couche décisionnelle et temps réel

L'ESP32-C3 interprète les données caméra et sélectionne la stratégie courante — recherche, approche ou ramassage. Il transmet ses ordres au **FPGA DE10-Lite** qui les exécute en temps réel :

- **Régulation PID** — asservissement en vitesse de chaque roue à partir des capteurs à effet Hall ; les deux boucles s'exécutent simultanément grâce au parallélisme natif du FPGA
- **Génération PWM** — signaux à 20 kHz pour le pont en H, avec gestion indépendante de la vitesse et du sens de rotation de chaque moteur
- **Acquisition ultrason** — pilotage des 8 capteurs HC-SR04 en parallèle et déclenchement de l'évitement obstacles sous 200 mm
- **Séquencement de la pince** — Machine à états gérant l'ouverture, la fermeture et la sécurité thermique du moteur pas à pas 28BYJ-48
- **Arrêt d'urgence matériel** — signal asynchrone KEY0 forçant l'extinction de tous les actionneurs en moins de 20 ns

4.2.3 Chaîne d'actionnement

Le FPGA pilote trois actionneurs distincts :

- **Moteurs de roues** — via un driver pont en H, asservis en boucle fermée PID grâce aux capteurs à effet Hall
- **Moteurs de bras** — déclenchés au franchissement du seuil BBOX_NEAR
- **Pince** — moteur pas à pas 28BYJ-48 en mode demi-pas (8 phases) pour maximiser le couple lors de la saisie

4.3 Outils et environnements de développement

4.3.1 FPGA — Intel Quartus Prime

La logique VHDL du DE10-Lite est développée sous **Intel Quartus Prime Lite**, certains modules ayant été simulés sous **ModelSim** avant intégration dans le **Top_Robot** pour valider chronogrammes et machines à états sans déploiement matériel systématique.

4.3.2 Microcontrôleurs — Arduino IDE

Les deux ESP32 sont programmés en **C/C++** sous **Arduino IDE** avec le BSP Seeed Studio. Le modèle FOMO est déployé sous forme de bibliothèque C++ générée par **Edge Impulse Studio** et embarquée directement dans le firmware de l'ESP32-S3.

4.3.3 Supervision PC — Python

Un script **Python** lit le port série de l'ESP32-C3 et trace en temps réel vitesse, angle et état du robot via **matplotlib**, permettant de diagnostiquer les dérives PID et valider les seuils de détection sans modifier le firmware.

4.3.4 Conception mécanique — Autodesk Fusion 360

Le châssis et les pièces mécaniques sont modélisés sous **Autodesk Fusion 360** (licence éducation). Le positionnement des composants électroniques et des capteurs est validé dans l'assemblage CAO avant toute réalisation physique.

5. Module de déplacement et motorisation

Le module de déplacement repose sur une chaîne de contrôle entièrement implémentée en VHDL sur le FPGA DE10-Lite, dont l'architecture est présentée en Figure 6. Une trame UART reçue de l'ESP32-C3 est décodée pour en extraire la consigne de vitesse. Celle-ci est comparée à la vitesse mesurée, fournie par le bloc **Frequency extractor** alimenté par le **Rotation counter** issu du capteur Hall. Le régulateur **PI** calcule la commande correctrice transmise au **Driver** BTS7960, qui alimente directement les moteurs. Les sections suivantes détaillent chacun de ces blocs.

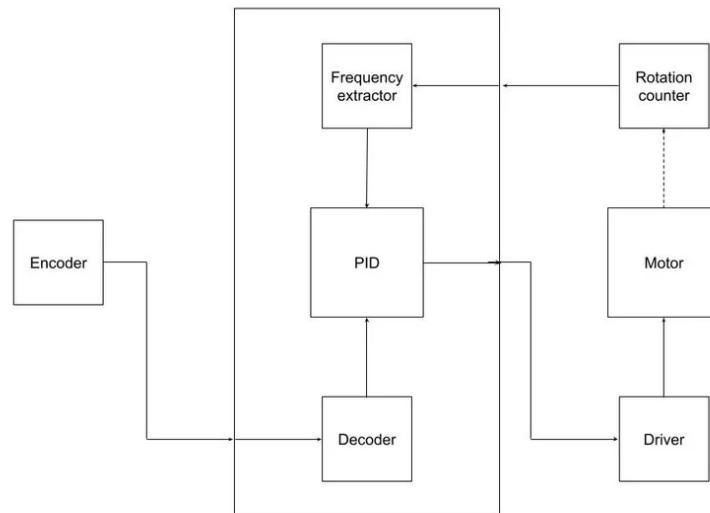


FIGURE 6 – Architecture de la chaîne de contrôle moteur sur FPGA

5.1 Choix et dimensionnement des actionneurs

5.1.1 Expression du besoin mécanique

Pour intervenir efficacement entre deux points et ne pas perturber le rythme du jeu, le robot doit atteindre une vitesse de pointe de **3 m/s** — exigence fixée par le cahier des charges afin de garantir une expérience de jeu fluide et continue. Avec des roues de diamètre $D = 75$ mm, la vitesse de rotation minimale requise est :

$$n_{\min} = \frac{60 \cdot v}{\pi \cdot D} = \frac{60 \times 3}{\pi \times 0,075} \approx 764 \text{ RPM}$$

En tenant compte des frottements et du rendement du réducteur ($\eta \approx 0,75$), le couple utile à délivrer est estimé à **supérieur à 1 N.m** par moteur.

5.1.2 Sélection et validation : JGB37-520 24 V 960 RPM

Le **JGB37-520 24 V — 960 RPM** satisfait l'ensemble des contraintes. Sa vitesse à vide produit une vitesse linéaire théorique :

$$v_{\max} = \frac{960 \times \pi \times 0,075}{60} \approx 3,77 \text{ m/s}$$

soit une marge de **+25%** au-dessus du seuil requis, absorbant les pertes en charge réelle. Son couple nominal de **1,5 N.m** dépasse le besoin estimé, sa masse de **160 g** reste compatible avec le gabarit du châssis, et son alimentation en **24 V** est assurée directement par le pack batterie 7S (25,9 V nominal).

5.2 Pièces imprimées dédiées aux actionneurs

5.2.1 Adaptateur roue-moteur

L'arbre de sortie du JGB37-520 présente une section cylindrique à méplat — un cylindre tronqué sur un quart de sa longueur — incompatible avec le logement hexagonal natif de la roue. Une pièce adaptatrice a donc été conçue et imprimée en **PETG** pour assurer la liaison entre les deux.

L'adaptateur reprend en négatif la géométrie exacte de l'arbre côté moteur, et présente un profil hexagonal côté roue, transmettant ainsi le couple sans jeu angulaire. Le PETG a été retenu pour sa rigidité, sa résistance aux contraintes mécaniques répétées et sa bonne tolérance dimensionnelle à l'impression.

5.2.2 Couronne magnétique sur l'arbre moteur

Pour mesurer la vitesse de rotation, une couronne d'aimants **N45** est fixée directement sur l'arbre moteur. Les aimants sont disposés en alternance de polarité Nord/Sud sur la circonférence, générant un signal périodique détectable par le capteur à effet Hall SS49E placé en regard.

Avec **6 aimants** répartis uniformément, la fréquence du signal Hall est directement proportionnelle à la vitesse de rotation :

$$f = \frac{n \times N_{aimants}}{60}$$

Le moteur opérant dans la plage $[0 ; 960]$ tr/min, la fréquence lue par le capteur évolue dans la plage :

$$f \in \left[\frac{0 \times 6}{60} ; \frac{960 \times 6}{60} \right] = [0 ; 96] \text{ Hz}$$

Cette plage de 96 Hz offre une granularité suffisante pour estimer la vitesse du robot avec une bonne précision sur l'ensemble de la plage de fonctionnement — y compris aux faibles régimes où l'asservissement PID est le plus sensible aux erreurs de mesure. Elle est par ailleurs largement compatible avec la fréquence d'échantillonnage du FPGA (50 MHz), garantissant une acquisition sans aliasing ni perte d'impulsion. La couronne a été imprimée en **PETG** avec des logements calibrés pour maintenir les aimants sans adhésif.

5.2.3 Support SS49E

Le capteur à effet Hall **SS49E** doit être maintenu à distance fixe et constante de la couronne magnétique pour garantir la répétabilité de la mesure. Un support dédié a été conçu et imprimé en **PETG** pour répondre à cette contrainte.

La pièce se compose d'une **platine de fixation** percée d'un trou de vissage permettant de solidariser l'ensemble au châssis moteur, et d'une **équerre triangulée** qui positionne la face active du SS49E perpendiculairement à l'axe de rotation. La triangulation assure la rigidité de l'équerre et supprime tout risque de flexion susceptible de modifier la distance capteur/aimants.

Le logement calibré pour le boîtier **TO-92** intègre trois fentes individuelles pour les broches, garantissant un positionnement angulaire reproductible. Le capteur est maintenu en position par du ruban adhésif appliqué sur sa face supérieure. La profondeur du logement est calculée pour placer la face active du capteur à **2,5 mm** de la surface des aimants, distance comprise dans la plage de fonctionnement optimal du SS49E et assurant un niveau de champ suffisant pour discriminer clairement les alternances Nord/Sud de la couronne.

5.3 Mesure de vitesse par capteur à effet Hall SS49E

5.3.1 Principe de mesure et choix du capteur

La mesure de vitesse repose sur la détection du champ magnétique alternatif généré par le passage successif de pôles Nord et Sud devant la face active d'un capteur à effet Hall linéaire **SS49E**. La fréquence du signal produit est directement proportionnelle à la vitesse de rotation de la roue — deux approches d'implémentation ont été étudiées et sont détaillées dans les sections suivantes.

Ce capteur, ratiométrique, délivre une tension de repos égale à la moitié de son alimentation ($V_0 \approx 2,5\text{ V}$ sous 5 V), qui croît ou décroît proportionnellement au champ perçu, avec une sensibilité de l'ordre de **1,4 à 1,8 mV/G**.

5.3.2 Première approche : circuit de conditionnement analogique (non conservé)

La première implémentation consistait à placer le SS49E directement contre le carter du moteur JGB37-520, en exploitant le champ des aimants du rotor interne. Le signal mesuré n'excédait pas **2,5 mVpp** — amplitude trop faible pour être exploitée directement par l'ADC du FPGA.

L'objectif de la chaîne de conditionnement était de produire en sortie un **signal crêteaux** dont chaque front — montant ou descendant — correspond à **un quart de tour de roue**, permettant au FPGA de comptabiliser les impulsions et d'en déduire la vitesse.

La chaîne conçue à cet effet comprend trois étages : prétraitement (gain 56), dérivation et mise en forme vers $3,3\text{ V}$.

Le circuit a été intégralement routé sous **PADS Logic et PADS Layout** puis fabriqué sur PCB (Figure 7). Les tests ont d'abord validé le fonctionnement de la carte en **simulant l'entrée SS49E par un générateur de fonctions** à $2,5\text{ mVpp}$: la chaîne délivrait bien un signal crêteaux propre en sortie. En revanche, le passage au capteur réel n'a jamais abouti malgré plusieurs jours de débogage — le bruit de fond et l'instabilité du signal source réel se sont révélés incompatibles avec le gain élevé de la chaîne.

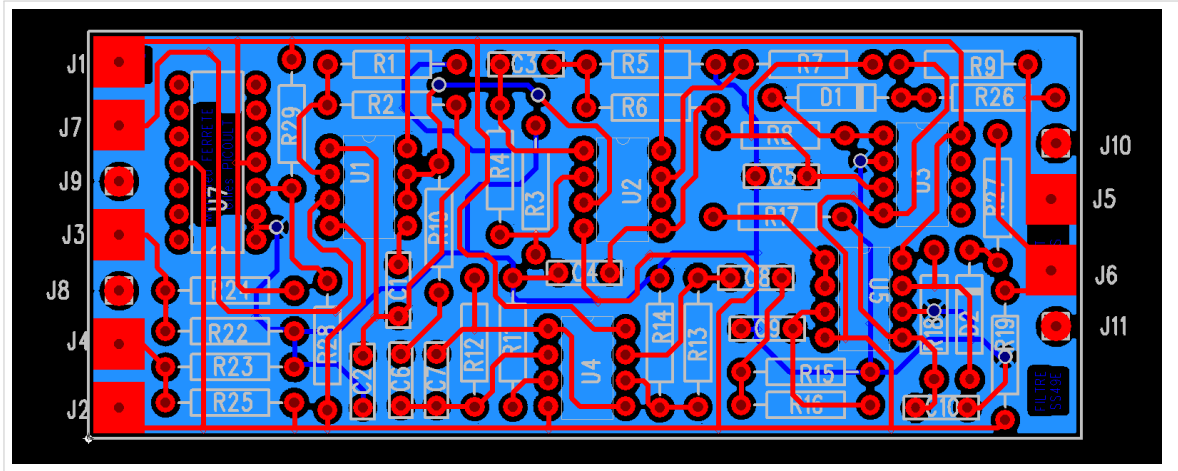


FIGURE 7 – Routage PCB du circuit de conditionnement — PADS Layout

Après trois semaines de développement, la décision d'abandon a été actée : l'architecture à couronne magnétique externe (section 5.2.2) s'affranchit du problème à la source en fournissant un signal d'amplitude suffisante, sans recourir à un conditionnement analogique de précision.

5.3.3 Solution retenue : acquisition ADC sur FPGA en VHDL

L'abandon du conditionnement analogique a conduit à une refonte complète de l'approche. La couronne magnétique externe (section 5.2.2) fournit un signal d'amplitude suffisante pour être **échantillonné directement par l'ADC 12 bits du DE10-Lite**, via l'IP dédiée du programme étudiant Quartus, sans aucune électronique de conditionnement externe.

• Phase d'initialisation : calibration automatique

Au démarrage, un processus VHDL `p_init` effectue une **calibration automatique du signal** sur 3 demi-cycles consécutifs. La machine à états `t_init_state` détecte successivement les maxima $V_{max,0..2}$ et minima $V_{min,0..2}$ du signal Hall en balayant les passages par la valeur médiane `c_ADC_MID` = 2048, confirmée sur `c_MID_CONFIRM` = 10 fronts consécutifs pour garantir la synchronisation.

Une fois `s_init_done` = '1', le module bascule en mode de suivi continu.

• Suivi de période : machine à états `p_main`

Le processus principal `p_main` implémente une **machine à états à 12 états** qui segmente chaque période du signal en ses points caractéristiques : 3 maxima, 3 minima et 6 passages par

V_{mean} . À chaque transition détectée — avec une marge de tolérance $c_ADC_MARGIN = 16$ LSB pour absorber le bruit résiduel — un **compteur de ticks 31 bits** est capturé puis réinitialisé. La période complète est reconstituée par sommation des 12 intervalles :

$$T_{période} = \sum_i \Delta t_i = tick_{period} \quad [\text{coups d'horloge à 50 MHz}]$$

La vitesse de rotation s'en déduit directement :

$$n = \frac{60 \times f_{clk}}{N_{aimants} \times T_{période}} = \frac{60 \times 50 \times 10^6}{6 \times tick_{period}} \quad [\text{tr/min}]$$

La sortie expose en temps réel la valeur de la dernière période actualisée.

• Transmission et validation expérimentale

Les données brutes ADC et la période mesurée sont transmises au PC via **UART**. Un script Python lit les trames ADC et affiche la vitesse de rotation de la roue en appliquant l'équation ci-dessus. La Figure 8 présente le signal obtenu à **8 V constant** : la vitesse mesurée - et non la fréquence - est **1,072 m/s**

Le signal oscille proprement entre les codes ADC 800 et 3100 (amplitude ≈ 2300 points sur 4095), confirmant que la couronne magnétique externe fournit un champ suffisant pour une mesure fiable sans aucun prétraitement analogique.

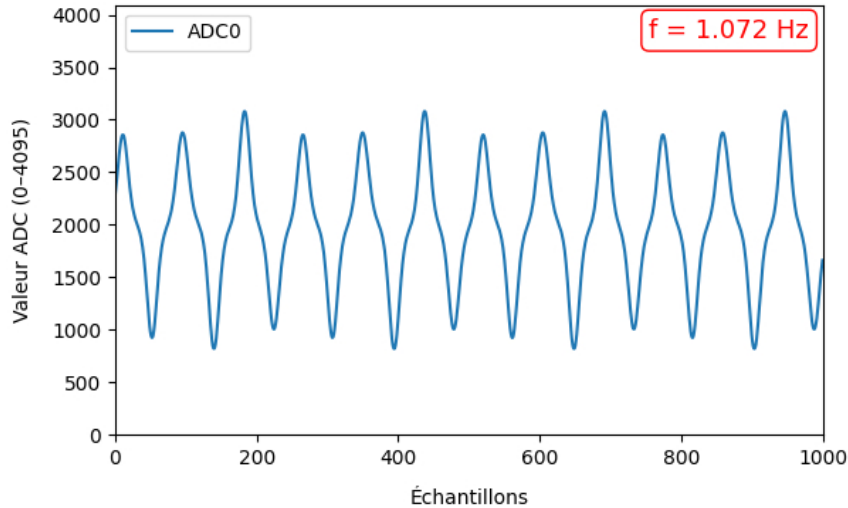


FIGURE 8 – Signal ADC du SS49E à tension moteur de 8 V

5.4 Contrôleur moteur BTS en VHDL

Le module `Motor_Controller` assure l'interface entre la logique de commande du FPGA et le driver de puissance **BTS7960**, qui alimente les moteurs de traction. Il reçoit en entrée une **consigne de vitesse sur 8 bits** (0–255) et un **bit de direction** issus du module PID (section 5.5), et génère en sortie les signaux PWM et EN du BTS7960. Une instance est dédiée à chaque roue, permettant un contrôle indépendant.

5.4.1 PWM

La consigne de vitesse est traduite en un signal PWM : un signal carré de période fixe dont le temps à l'état haut varie proportionnellement à la consigne. La fréquence du PWM de **20 kHz** est retenue car elle est inaudible, garantit un lissage efficace du courant et reste dans les limites de commutation du BTS7960.

5.4.2 Arrêt d'urgence

Lors d'un reset - déclenché par la détection d'un obstacle - les signaux PWM et EN sont immédiatement forcés à '0', coupant l'alimentation des roues instantanément .

5.5 Asservissement en vitesse : régulateur PI en VHDL

Le module d'asservissement constitue le cœur du contrôle en boucle fermée du déplacement. Son rôle est d'annuler en permanence l'écart entre la **vitesse consigne** — reçue de l'ESP32-C3 via UART — et la **vitesse mesurée** issue du module `adc_period` (section 5.3.3). La sortie pilote directement le module `Motor_Controller` (section 5.4), qui traduit cette commande en signal PWM vers le driver moteur BTS7960.

5.5.1 Architecture : deux régulateurs PI indépendants

Deux instances du régulateur sont déployées en parallèle sur le FPGA — une par roue. Cette indépendance est indispensable : en imposant des consignes différentes à chaque instance, l'ESP32-C3 orchestre l'ensemble des manœuvres du robot (ligne droite, virage, pivot, arrêt) sans logique supplémentaire côté FPGA.

Chaque régulateur implémente une loi **Proportionnelle-Intégrale** :

$$u[n] = K_p \cdot e[n] + K_i \cdot \sum_{k=0}^n e[k] \quad \text{avec} \quad e[n] = v_{\text{consigne}}[n] - v_{\text{mesurée}}[n]$$

Le terme proportionnel réagit immédiatement à l'erreur courante ; le terme intégral accumule les erreurs passées et élimine l'écart statique résiduel, garantissant que la vitesse mesurée rejoint exactement la consigne en régime permanent.

5.5.2 Limitation et anti-windup

La sortie $u[n]$ est saturée sur la plage admissible par le `Motor_Controller` (0–255). Un mécanisme d'**anti-windup** gèle l'accumulation intégrale dès que la commande atteint la saturation, évitant toute dérive de l'intégrateur lors des phases d'accélération maximale ou d'arrêt d'urgence.

5.5.3 État d'avancement

Ce module a été conçu et dimensionné sur le plan théorique — architecture PI, gestion de l'anti-windup, interfaçage avec `adc_period` et `Motor_Controller` — mais n'a pas été implémenté ni simulé dans le temps imparti. Le robot a donc opéré en **boucle ouverte** lors des tests finaux, la consigne UART étant transmise directement au `Motor_Controller` sans correction par retour capteur.

5.6 Stratégie de déplacement et navigation

La stratégie de navigation définit comment le robot parcourt le terrain pour maximiser la couverture et collecter les balles efficacement. Cette partie a été conçue et documentée mais n'a pas été implémentée faute de temps.

5.6.1 Stratégies étudiées

Deux parcours complémentaires ont été étudiés :

- **Parcours en O** : le robot décrit des rectangles concentriques depuis les bords vers le centre. Couverture progressive du terrain, demi-tours limités, compatible avec une opération en continu durant le match. Retenu comme **stratégie principale**.
- **Parcours en 8** : extension du parcours en O intégrant un franchissement du filet central. Permet de traiter les balles échouées en zone filet, inaccessibles par un simple parcours rectangulaire. Envisagé comme **stratégie secondaire** après validation du O.

Une **localisation par fingerprinting Wi-Fi** avait également été étudiée : en mesurant la puissance des réseaux environnants (RSSI), le robot aurait pu estimer sa position absolue sur le court sans infrastructure dédiée, exploitant le module Wi-Fi natif de l'ESP32-C3.

5.6.2 Stratégie retenue : parcours en O puis en 8

Au démarrage, le robot effectue un **premier tour complet en O** à vitesse réduite afin de détecter les bords du terrain et calibrer ses dimensions. Une fois cette phase de cartographie

terminée, il bascule en **mode recherche** : parcours en O resserré pour la surface principale, complété ponctuellement par le parcours en 8 pour traiter la zone filet. Le parcours en O présente trois avantages décisifs : il minimise les demi-tours, maintient le robot en périphérie (loin des joueurs) en début de séquence, et permet une reprise immédiate après un évitement d'obstacle sans recalcul de trajectoire.

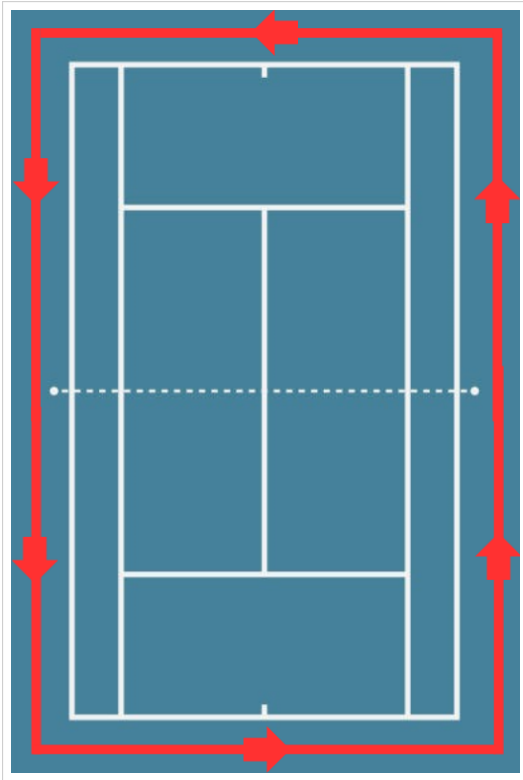


FIGURE 9 – *Parcours en O — balayage rectangulaire concentrique*

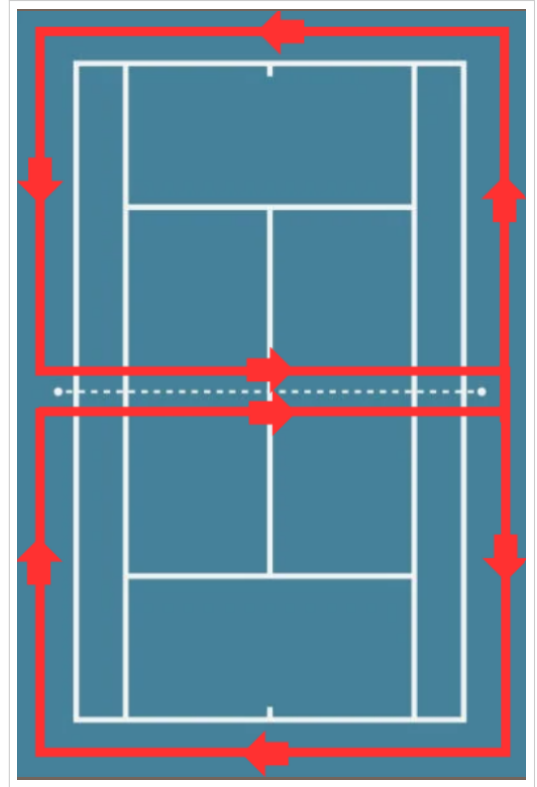


FIGURE 10 – *Parcours en 8 — traitement de la zone filet*

5.6.3 Implémentation envisagée

L'implémentation aurait reposé sur l'ESP32-C3, maintenant un **état de navigation courant** parmi : CALIBRATE, SEARCH, APPROACH, COLLECT et AVOID. Les consignes de vitesse et de direction auraient été transmises au FPGA via UART, exploitant les modules `Motor_Controller` et le régulateur PI (section 5.5). La détection d'un obstacle ultrason à moins de 200 mm aurait déclenché un pivot sur place avant reprise du parcours ; la détection d'une balle par la caméra (section 6.2) aurait interrompu le balayage pour basculer en mode **APPROACH**.

6. Module de détection et navigation

6.1 Détection d'obstacles par ultrasons

Le HC-SR04 est retenu pour la détection d'obstacles — grillages, filets, parois — grâce à son insensibilité aux conditions lumineuses et à son protocole d'interfaçage minimal : une impulsion **TRIG** de 10 μs déclenche l'émission de 8 impulsions ultrasonores à **40 kHz** ; la largeur du signal **ECHO** en retour est directement proportionnelle à la distance. Sa plage de mesure utile s'étend de **20 mm à 4 000 mm**, ce qui couvre l'intégralité des distances d'évitement nécessaires sur un court de tennis.

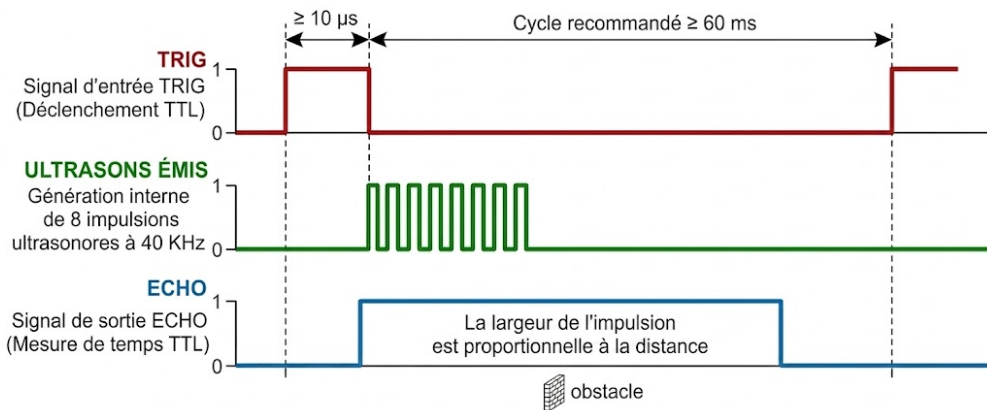


FIGURE 11 — Chronogramme de fonctionnement du HC-SR04

6.1.1 Conversion temps $\rightarrow$ distance

Le driver VHDL implémente un compteur synchrone activé sur le front montant d'**ECHO** et arrêté sur le front descendant. Avec une horloge de **50 MHz** (période 20 ns) et la vitesse du son de 340 m/s, la conversion en millimètres s'écrit :

$$d \text{ (mm)} = \frac{N_{\text{echo}} \times 20 \times 10^{-9} \times 340}{2 \times 10^{-3}} = \frac{N_{\text{echo}}}{294,12}$$

Le facteur 2 au dénominateur traduit le trajet aller-retour de l'onde. Un **timeout** à 20 ms (1 000 000 cycles) est imposé : si aucun écho n'est reçu, la distance est forcée à zéro pour signaler l'absence d'obstacle ou un capteur défaillant, et éviter tout gel de la machine à états.

6.1.2 Adaptation électrique : pont diviseur de tension

Le HC-SR04 délivre un signal ECHO à **5 V** alors que les GPIO de la DE10-Lite (Intel MAX10) tolèrent **3,3 V**. Injecter directement ce signal risquerait de claquer la broche voire le banc d'E/S entier. Un pont diviseur R1/R2 abaisse la tension sans composant actif :

$$V_{\text{out}} = V_{\text{in}} \times \frac{R_1}{R_1 + R_2} = 5 \times \frac{3,3 \text{ k}\Omega}{3,3 \text{ k}\Omega + 1,8 \text{ k}\Omega} \approx 3,24 \text{ V}$$

Cette valeur respecte le standard LVCMOS 3,3 V : supérieure au seuil logique haut, inférieure à la tension maximale des broches. Le signal TRIG, émis par le FPGA à 3,3 V, est quant à lui directement compatible avec le niveau bas du HC-SR04.

6.1.3 Placement sur le robot

Le cône de détection utile du HC-SR04 est limité à **15°–30°**. Une configuration à 4 capteurs (avant/arrière/gauche/droite) laisserait des zones aveugles de 60° à chaque angle. La solution retenue est une couronne de **8 capteurs espacés de 45°**, assurant le chevauchement des cônes et une couverture omnidirectionnelle. Chaque capteur étant piloté par un module VHDL indépendant, le parallélisme natif du FPGA permet de traiter les 8 mesures simultanément sans overhead temporel.

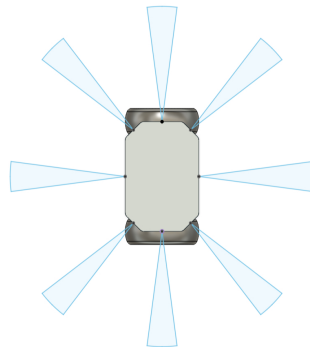


FIGURE 12 – *Disposition des 8 capteurs HC-SR04 sur le robot*

6.2 Détection des balles par modèle d'apprentissage

6.2.1 Matériel : XIAO ESP32-S3 Sense

Le module de vision repose sur le **XIAO ESP32-S3 Sense** de Seeed Studio. Ce module intègre un microcontrôleur ESP32-S3 cadencé à **240 MHz**, une caméra **OV3660** en résolution QVGA (320×240 pixels), **8 Mo de Flash** et **8 Mo de PSRAM** — indispensable pour stocker les buffers d'image, le microcontrôleur ne disposant pas de RAM interne suffisante pour traiter des images. L'interface **UART** assure la communication avec la DE10-Lite.

Avant l'intégration du module de détection, un **serveur vidéo de diagnostic** a été développé : l'ESP32-S3 est configuré en point d'accès Wi-Fi et diffuse un flux MJPEG consultable depuis un navigateur. Cette étape a permis de valider la netteté de l'image, régler la mise au point et vérifier le cadrage avant tout développement logiciel.

6.2.2 Première approche : filtre de couleur HSV (abandonnée)

La première stratégie de détection consistait à isoler la balle par seuillage colorimétrique dans l'espace **HSV** (Teinte-Saturation-Valeur), en ciblant la plage jaune-vert caractéristique d'une balle de tennis : $H \in [28, 48]$.

Lors des essais physiques, un défaut bloquant est apparu : la largeur détectée en pixels refusait de décroître linéairement avec la distance. À 20 cm la caméra mesurait 220 px ; à 40 cm elle mesurait encore 140 px au lieu des 110 px théoriques. Deux phénomènes physiques l'expliquent :

- **Blooming** : la surface feutrée de la balle réfléchit fortement la lumière — les photons excédentaires débordent sur les pixels adjacents, créant un halo jaune artificiellement large.
- **Variabilité des conditions** : les valeurs HSV calibrées dans un environnement (H 28, S 120, V 140) devenaient invalides sous un éclairage différent, et des fonds de couleur similaire (herbe verte, sol clair) généraient de nombreux faux positifs.

Décision : abandon complet de l'approche HSV au profit d'un modèle de deep learning, insensible aux variations d'éclairage.

6.2.3 Modèle d'apprentissage sur Edge Impulse

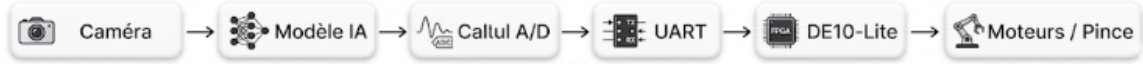


FIGURE 13 – *Architecture globale du système de détection et de ramassage*

Architecture FOMO Le modèle de détection est développé sur **Edge Impulse Studio** avec l'architecture **FOMO** (*Faster Objects, More Objects*), spécifiquement conçue pour les microcontrôleurs sans GPU : empreinte mémoire minimale, temps d'inférence court, compatible ESP32-S3.

Dataset et entraînement Le dataset est constitué de **100 photographies de balles de tennis** capturées dans des conditions variées — différentes distances, orientations, fonds et éclairages — comme l'illustre la figure 14. Chaque image est annotée avec un rectangle de délimitation. Le modèle final atteint un **F1-Score de 100 %** sur le jeu de validation.



FIGURE 14 – *Dataset d'entraînement — balles sur fonds variés*

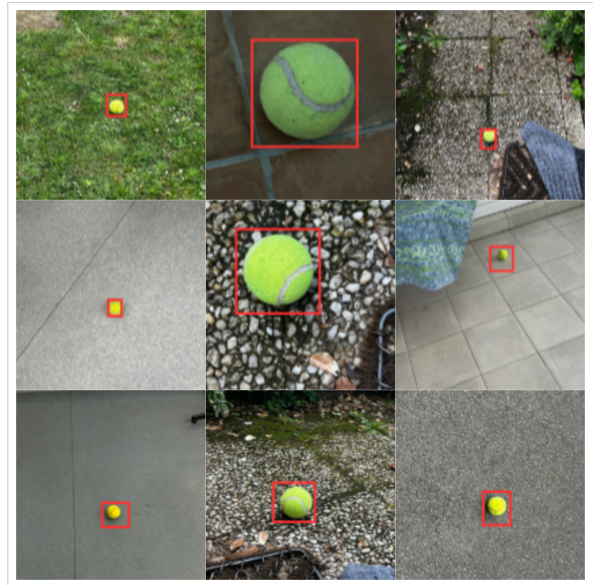


FIGURE 15 – *Résultats FOMO — bounding boxes détectées*

Tentative SenseCraft (non retenue) Un modèle pré-entraîné de détection de pommes, disponible sur SenseCraft AI, avait initialement été testé pour sa morphologie sphérique similaire. Si la confiance affichée dépassait **85 %** en conditions contrôlées, la solution a été abandonnée : la

couleur jaune-vert de la balle est trop éloignée du rouge-vert des pommes, et les autres modèles disponibles sur la plateforme n'étaient pas compatibles avec l'ESP32-S3.

6.2.4 Traitement des données et sélection de la cible

Lorsque plusieurs balles sont détectées simultanément, un **algorithme de scoring** sélectionne la cible prioritaire selon deux critères combinés :

$$\text{score} = l_{\text{bbox}} - d_{\text{centre}}$$

où l_{bbox} est la largeur de la boîte de détection en pixels (grande = proche) et d_{centre} est la distance euclidienne entre le centre de la balle et le centre de l'image. Un score élevé désigne une balle grande et centrée — donc proche et déjà dans l'axe du robot.

Un **filtre de lissage exponentiel** (EMA, $\alpha = 0,1$) est appliqué sur la taille de la bounding box pour éliminer les oscillations d'une inférence à l'autre :

$$T_{\text{filt}}[n] = \alpha \cdot T_{\text{raw}}[n] + (1 - \alpha) \cdot T_{\text{filt}}[n - 1]$$

Avec $\alpha = 0,1$, le robot n'accorde que 10 % de confiance à la nouvelle mesure et conserve 90 % de l'historique, immunisant l'affichage contre le bruit du capteur.

6.2.5 Estimation de la distance et de l'angle

Angle de rotation La position horizontale de la balle en pixels est convertie en angle physique selon :

$$\theta = \frac{x_{\text{balle}} - L/2}{L/2} \times \theta_{\text{FOV}/2}$$

avec $L = 96$ px (largeur FOMO). Si la balle est détectée à $x = 72$ px, le décalage est de +24 px vers la droite, ce qui fournit directement la consigne de rotation transmise à la DE10-Lite.

Estimation de la distance La modélisation par sténopé (théorème de Thalès) a été testée mais s'est révélée inutilisable en raison de la distorsion de l'objectif grand-angle et des oscillations de ± 40 % sur la bounding box à position fixe. La solution retenue est une **approche par seuils directs** calibrés expérimentalement : deux seuils `BBOX_FAR` et `BBOX_NEAR` définissent les zones d'approche et de déclenchement du ramassage, offrant une robustesse nettement supérieure.

6.2.6 Transmission des coordonnées vers la DE10-Lite

À chaque inférence (période : **1,2 s**), l'ESP32-S3 transmet via **UART** un paquet contenant l'angle de rotation calculé et le seuil de distance estimé. La DE10-Lite exploite ces informations pour piloter les moteurs de déplacement : l'angle corrige le cap du robot, tandis que le franchissement d'un seuil de taille de bounding box déclenche la séquence de ramassage.

Paramètre	Valeur
Plateforme d'entraînement	Edge Impulse Studio
Architecture	FOMO MobileNetV2
Dataset	100 images annotées
F1-Score (validation)	100 %
Résolution d'entrée	96×96 pixels
Période d'inférence	1,2 s
Filtre EMA (α)	0,1
Estimation de distance	Seuils l_{bbox} calibrés

TABLE 2 – *Récapitulatif des paramètres du module de détection*

7. Conception mécanique et CAO

L'intégralité de la structure mécanique de R2B2 a été modélisée sous **Autodesk Fusion 360** avant toute fabrication. Les Figures 16 et 17 présentent deux vues de l'assemblage final, illustrant l'organisation spatiale des sous-systèmes détaillée dans les sections suivantes.

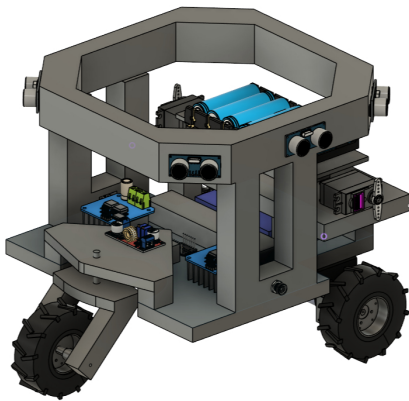


FIGURE 16 – Vue CAO — face arrière

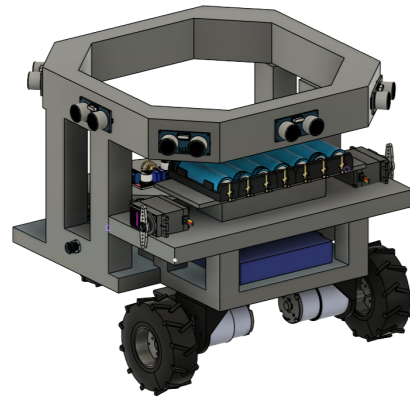


FIGURE 17 – Vue CAO — face avant

7.1 Structure et châssis

La conception du châssis a été intégralement modélisée sous **Autodesk Fusion 360** avant toute fabrication physique. Cette étape de CAO a permis de visualiser le placement réel de chaque composant à l'échelle, d'anticiper les encombrements et de valider les cotes avant la découpe du bois.

7.1.1 Conception et choix des matériaux

Le châssis est réalisé en **bois**, matériau offrant un excellent compromis entre légèreté, rigidité et facilité d'usinage. Il permet la réalisation de pièces sur mesure par découpe simple, tout en supportant l'ensemble des composants électroniques et mécaniques sans déformation sous charge.

7.1.2 Architecture multi-niveaux

Le robot est organisé sur **trois niveaux distincts**, optimisant l'espace disponible et facilitant l'accès aux composants selon leur fréquence de manipulation

- **Niveau inférieur — base motrice** : accueille les roues motrices et la roue folle. La carte **FPGA DE10-Lite** est installée juste au-dessus de cette base : une fois câblée, elle ne nécessite aucune intervention régulière.
- **Niveau intermédiaire — énergie** : plateau central dédié au **pack batterie 7S1P**. Son positionnement volontairement accessible permet de manipuler les cellules sans démontage partiel du robot.
- **Niveau supérieur — anneau octogonal** : structure à huit côtés conçue spécifiquement pour l'intégration des **capteurs ultrasons HC-SR04**, orientés sous différents angles pour la couverture 360°. Les faces présentent une **inclinaison de 7,5°** vers le haut, évitant que les ondes ultrasoniques ne rebondissent sur le sol et ne génèrent de fausses détections à grande portée.

7.1.3 Répartition des masses et ergonomie

L'ensemble des composants est disposé de façon **symétrique** par rapport au centre du robot, garantissant une répartition homogène du poids et une stabilité optimale lors des changements de direction. Dans un souci d'ergonomie, l'**interrupteur général** et le **port de charge** sont positionnés en façade de la structure, accessibles directement de l'extérieur sans démontage.

7.2 Bras de ramassage : cinématique et actionnement

Le bras de ramassage constitue l'effecteur terminal du robot. Son rôle est de se déployer au sol pour saisir une balle détectée par la caméra, puis de la ramener dans le réservoir embarqué.

7.2.1 Problème cinématique

Le système retenu est un **bras à deux segments articulés** ($bras_1$ et $bras_2$) à deux degrés de liberté, positionné de chaque côté du robot. L'extrémité de $bras_2$ porte la pince. La trajectoire de cette extrémité doit satisfaire deux contraintes géométriques simultanées : atteindre le sol à l'avant du robot pour saisir la balle, et franchir le niveau supérieur du châssis sans collision avec le module caméra lors de la rétraction.

7.2.2 Courbes paramétriques

En notant L_1 et L_2 les longueurs des deux segments, θ_1 l'angle entre le sol et $bras_1$, θ_2 l'angle entre $bras_1$ et $bras_2$, $h = 18,8$ cm, $l = 21$ cm la hauteur de l'axe de rotation et α l'inclinaison globale, la position de la pince s'écrit :

$$\begin{cases} x = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{cases}$$

En imposant la contrainte $\theta_2 = \pi - 2\theta_1$, le problème se ramène à un seul paramètre $t = \theta_1$:

$$\begin{cases} x = (L_1 - L_2) \cos(t) \cos \alpha - (L_1 + L_2) \sin(t) \sin \alpha + l \\ y = (L_1 - L_2) \cos(t) \sin \alpha + (L_1 + L_2) \sin(t) \cos \alpha + h \end{cases}$$

La solution satisfaisant les contraintes est obtenue pour $L_1 = 10$ cm, $L_2 = 24$ cm et $\alpha = 8$, dont la trajectoire résultante est représentée Figure 18.

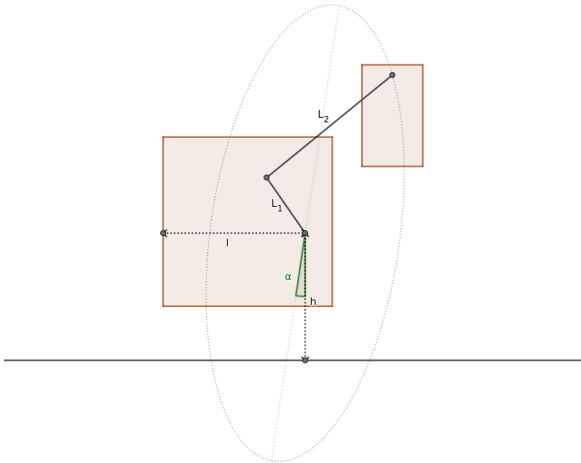


FIGURE 18 – Courbe paramétrique de la trajectoire de la pince

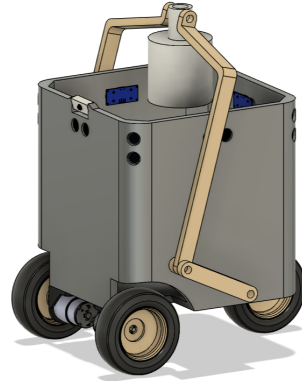


FIGURE 19 – Modélisation CAO du robot et ses bras articulés (version 3D précédente)

7.2.3 Actionnement

Le couple requis pour actionner les bras a été estimé supérieur à **1 N.m**. Le servomoteur **MG996R** a été retenu, offrant un couple de 1,08 N.m à 4,8 V — soit une marge de 20% — pour un coût réduit.

7.2.4 État d'avancement

Le bras de ramassage a été conçu et dimensionné sur le plan théorique — cinématique, choix des dimensions, validation de la trajectoire — mais n'a pas pu être traité dans le temps imparti. Aucune pièce n'a été modélisée sous Fusion 360 ni imprimée. Cette partie reste une perspective d'évolution directe du projet.

7.3 Pince de saisie

La pince constitue l'organe de préhension terminal, portée par l'extrémité du bras articulé. Son rôle est de s'ouvrir en position basse, d'enserrer la balle au sol, puis de se refermer pour la maintenir pendant la rétraction jusqu'au réservoir.

7.3.1 Solutions étudiées et choix

Quatre mécanismes ont été comparés : pelletage, aspiration, encerclement et pince. La solution par **pince robotique** a été retenue pour sa précision de saisie, sa fiabilité même en présence de plusieurs balles proches, et sa capacité à opérer indépendamment du déplacement du robot.

7.3.2 Principe mécanique — transmission par vis-écrou

Le mécanisme repose sur une **conversion rotation $\rightarrow$ translation** assurée par une vis centrale entraînée par le moteur. Un **écrou mobile** solidaire d'une pièce centrale se déplace verticalement le long de cette vis : vers le bas pour l'ouverture, vers le haut pour la fermeture. Ce coulisseau central est relié aux **bras articulés** de la pince par des biellettes ; son déplacement vertical est ainsi converti en mouvement symétrique d'écartement ou de rapprochement des mâchoires. L'ensemble des pièces a été modélisé sous Fusion 360 et imprimé en **PETG**.



FIGURE 20 – Pince — Modèle 3D

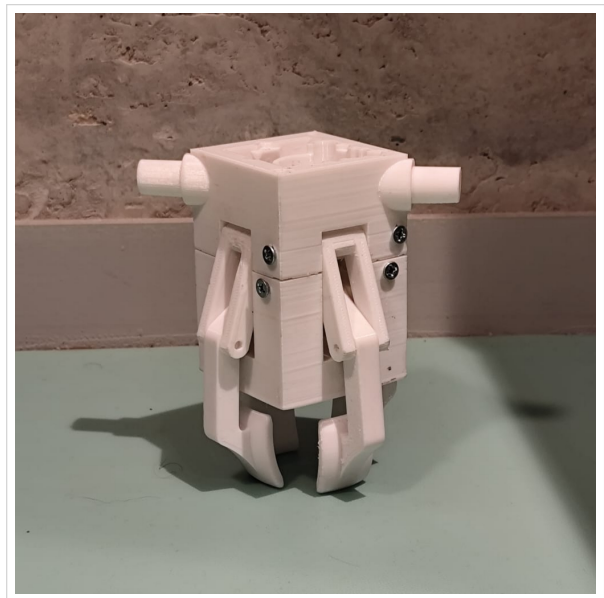


FIGURE 21 – Pince assemblée — position fermée

7.3.3 Actionnement — 28BYJ-48 et driver ULN2003

La vis centrale est entraînée par un **moteur pas à pas 28BYJ-48**, piloté via le driver **ULN2003**. Le mode **demi-pas** (8 phases) est utilisé pour maximiser le couple disponible lors de la saisie et affiner la résolution angulaire. La commande est gérée par une machine à états implémentée sur le FPGA : `IDLE` → `OPEN` → `WAIT BALL` → `CLOSE` → `HOLD`. Une protection logicielle coupe l'alimentation du moteur dès que la position fermée est atteinte, évitant toute surchauffe par blocage mécanique.

7.3.4 État d'avancement

La pince a été intégralement modélisée sous Fusion 360 et imprimée en PETG. Le mécanisme vis-écrou et l'articulation des mâchoires ont été validés mécaniquement. L'intégration dans la chaîne complète — déclenchement automatique depuis la détection caméra et remontée par le bras — n'a pas été finalisée faute de temps.

7.4 Roue folle

La roue folle assure le troisième point d'appui du robot, à l'arrière du châssis. Elle doit permettre une rotation libre dans toutes les directions pour ne pas contraindre les trajectoires imposées par les deux roues motrices.

7.4.1 Conception initiale et problème identifié

La première version positionnait l'axe de pivotement **directement sous la roue**, en alignement vertical avec son centre. Cette configuration, intuitivement logique, s'est révélée instable en pratique : lorsque le robot amorce un virage, la roue tend à osciller et à résister au changement de direction plutôt que de s'y aligner naturellement.

7.4.2 Solution retenue — axe déporté

L'étude des roues folles disponibles sur le marché a mis en évidence un principe commun à toutes les solutions fonctionnelles : **l'axe de pivotement est systématiquement déporté en avant de la roue**, et non centré sur elle. Ce décalage crée un bras de levier qui force la roue à s'aligner passivement dans le sens du déplacement, exactement comme la roulette d'un caddie ou d'une chaise de bureau. Plus le robot avance vite, plus l'alignement est automatique et stable.

La roue folle de R2B2 a donc été reconçue avec cet axe déporté. Elle est conçue en bois.

8. Système de batteries et gestion d'énergie

8.1 Pack batterie et régulation

Le système est alimenté par un pack **7S1P** composé de cellules **Samsung INR18650-25R**, offrant une tension nominale de **25,9 V** (29,4 V en charge complète, coupure à 17,5 V) pour une capacité de **2 500 mAh**, soit une énergie totale de **64,75 Wh**. Ce pack délivre un courant de décharge continu de **20 A**, suffisant pour absorber les pics de consommation des moteurs en accélération.

Un **BMS 7S** (Battery Management System) assure la protection active du pack : équilibrage des cellules à la charge, protection contre la surcharge, la surdécharge et les courts-circuits. Des régulateurs de tension abaissent la tension du pack vers les différents domaines d'alimentation du système (5 V pour les capteurs et cartes, 3,3 V pour le FPGA).

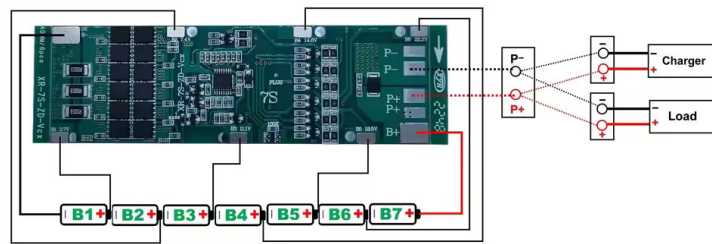


FIGURE 22 – BMS 7S — protection et équilibrage du pack batterie

8.2 Circuit de détection de seuils

Pour prévenir toute extinction brutale du système lors des pics de consommation moteurs et signaler l'état de charge au robot, un **circuit détecteur de seuils** à base de comparateurs a été conçu et réalisé sur PCB.

Deux seuils sont surveillés en continu via un pont diviseur ramenant la tension du pack vers une plage lisible par les comparateurs :

- **Seuil haut** — **29,4 V** : une **LED bleue** indique que le pack est chargé et la tension optimale pour les performances nominales

- **Seuil critique** — **21,5 V** : une **LED rouge** signale l'approche de la limite de décharge profonde, imposant un retour immédiat à la base pour recharge.

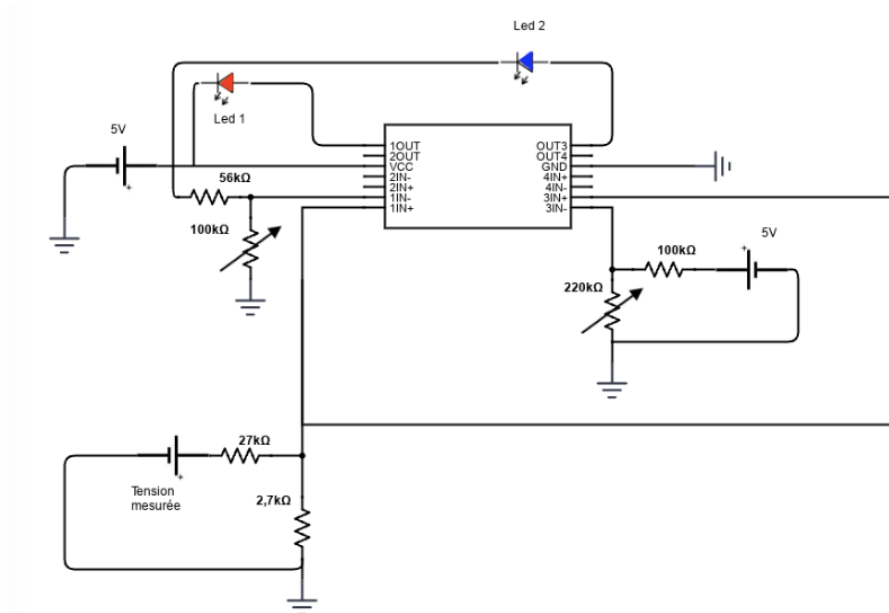


FIGURE 23 — *Circuit détecteur de seuils — indicateurs LED de charge*

La tension de référence des comparateurs est fixée par un pont diviseur résistif dimensionné pour correspondre exactement aux deux seuils. Ce circuit opère indépendamment du FPGA et du microcontrôleur — sa logique est **purement analogique**, garantissant son fonctionnement même en cas de défaillance logicielle.

9. Supervision et interface opérateur

La chaîne de supervision regroupe l'ensemble des modules permettant de transmettre les données du robot vers un poste de contrôle, de les visualiser en temps réel et d'assurer le suivi de son état opérationnel.

9.1 Transmission UART — Trame structurée

Les données acquises par le FPGA (ADC 12 bits) sont encapsulées dans une trame UART avant transmission série. Le module `uart_framer.vhd` implémente ce protocole de trame selon le format suivant :

Octet 0	Octet 1	Octets 2..N+1	Octet N+2
En-tête 0xAA	Longueur N	Données	Checksum XOR

Le checksum est calculé par XOR de tous les octets de données, permettant une détection simple d'erreurs de transmission. La FSM interne distingue les phases `BUILD` → `LOAD_BYTE` → `SEND_BYTE` → `WAIT_BYTE_END`, garantissant que les données sont stables avant chaque impulsion d'envoi.

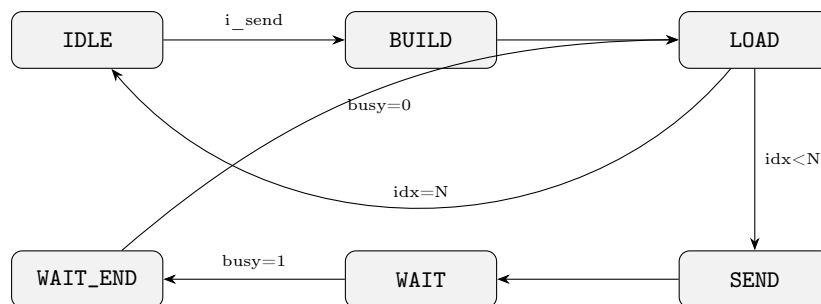


FIGURE 24 – FSM de `uart_framer.vhd`

9.2 Multiplexage UART — Deux canaux ADC

Le module `uart_mux.vhd` orchestre l'envoi alterné des deux canaux ADC (`i_ch0` et `i_ch1`) via une instance unique de `uart_framer`. La FSM `LOAD` → `SEND` → `WAIT_START` → `WAIT_END` garantit que la capture du canal actif (cycle 1) précède toujours le strobe d'envoi (cycle 2), évitant toute métastabilité sur les données. Une nouvelle trame est déclenchée dès que la précédente est terminée, maximisant le débit utile.

9.3 Visualisation série — Python Matplotlib

Côté PC, un script Python lit le flux série à **115 200 bauds** et affiche les deux canaux ADC en temps réel via `matplotlib.animation`. La fenêtre conserve les **1 000 derniers points** par canal. Un bouton *Pause* vide le buffer d'entrée en continu pour éviter l'accumulation de données pendant l'arrêt de l'affichage. Le canal ADC1 est utilisé pour remonter la **fréquence moteur** en appliquant la relation :

$$f = \frac{\pi \cdot D \cdot f_{clk}}{V_{ADC} \cdot N_{steps}}$$

où $D = 75$ mm est le diamètre de roue, $f_{clk} = 50$ kHz la fréquence de référence et $N_{steps} = 8192$ le nombre de pas par tour.

9.4 Interface de supervision — Tennis Robot Monitor

Une interface graphique complète a été développée en **Python/Tkinter** pour la supervision du robot. L'écran est divisé en deux zones principales (Figure 25) :

- **Panneau gauche** : télémétrie en temps réel (position x/y , vitesse, cap, balles collectées, niveau de batterie avec barre de progression colorée) et vue locale radar affichant les 8 capteurs ultrasons avec code couleur par distance (rouge < 1 m, orange 1–4 m, vert > 4 m).
- **Panneau droit** : vue terrain complète à l'échelle du court de tennis ($23,77 \times 10,97$ m), incluant une *occupancy map* mise à jour en temps réel par fusion des mesures ultrason et le trajet parcouru par le robot.

L'architecture repose sur un **adaptateur de source** qui bascule automatiquement entre données BLE réelles (module `ble_robot`) et un mode **démonstration** générant des trajectoires et distances simulées, permettant de tester l'interface sans robot physique.

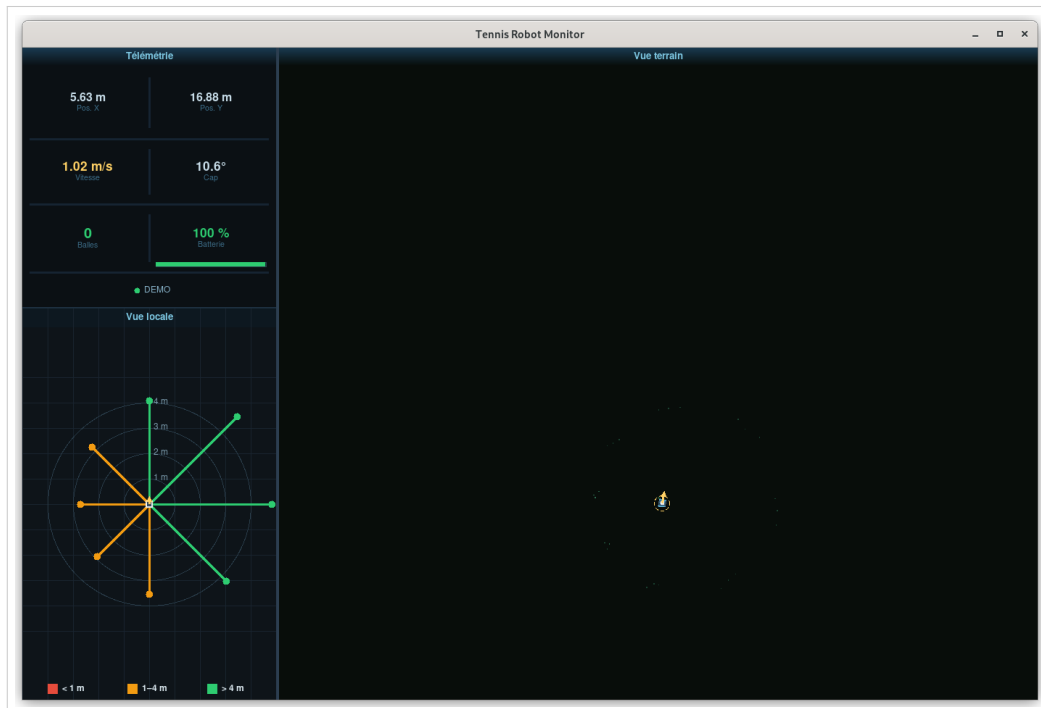


FIGURE 25 — Interface Tennis Robot Monitor — vue terrain et télémétrie

9.5 État d'avancement

Les modules VHDL `uart_framer` et `uart_mux` ont été validés en simulation et testés sur cible FPGA. Le script Python de visualisation série a été utilisé en banc de test pour vérifier les trames ADC. L'interface Tkinter est fonctionnelle en mode démonstration ; l'intégration avec le module BLE réel n'a pas été finalisée faute de temps.

10. Gestion

10.1 Gestion du temps — Diagramme de Gantt

Le diagramme de Gantt ci-dessous présente le déroulement du projet sur l'année.

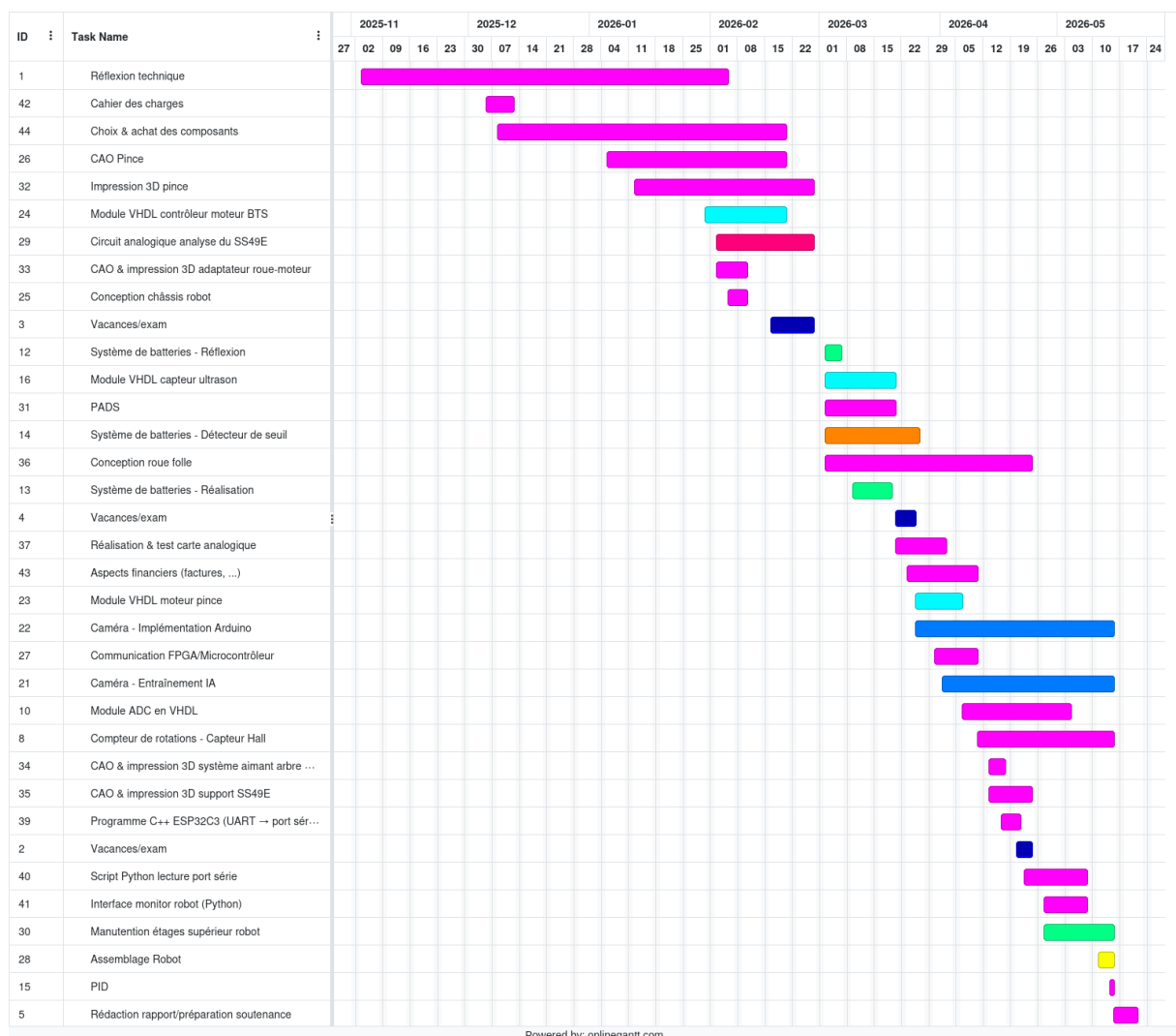


FIGURE 26 — Diagramme de Gantt — planification et suivi du projet R2B2

Couleur	Responsable(s)
	Matthieu
	Louis
	Mathilde
	Mathilde & Matthieu
	Alana & Théo
	Tom
	Louis & Matthieu
	Vacances / Examens

TABLE 3 – *Légende du diagramme de Gantt*

10.2 Gestion financière

Voici le récapitulatif des dépenses :

Catégorie	Désignation	Fournisseur	Qté	P.U.	Total	Date
Détection	Capteurs ultrason HC-SR04 (3,3–5 V)	AliExpress	8	1,20 €	9,09 €	07/01/2026
Motorisation	Roues RC caoutchouc hexagonales (×4)	AliExpress	1 lot	—	4,53 €	07/01/2026
	Moteur JGB37-520 960 RPM 24 V + support (remboursé)	AliExpress	1	8,39 €	7,94 €	07/01/2026
	Moteur JGB37-520 960 RPM 24 V + support	AliExpress	1	2,89 €	2,03 €	07/01/2026
	Servo MG996R 13 kg (×2) (bras)	AliExpress	1 lot	7,79 €	7,37 €	07/01/2026
	Moteur pas-à-pas 28BYJ-48 + driver ULN2003	AliExpress	1	2,20 €	1,86 €	29/01/2026
	Moteur JGB37-520 960 RPM 24 V + support	AliExpress	1	8,07 €	8,07 €	29/01/2026
	Moteur pas-à-pas 28BYJ-48 + ULN2003 (remplacement)	AliExpress	1	1,99 €	1,99 €	25/03/2026
Capteurs	Capteurs effet Hall SS49E / A3144 (×10)	AliExpress	1 lot	1,35 €	1,14 €	29/01/2026
	Micro servo SG90 (orientation caméra)	AliExpress	1	1,35 €	1,14 €	29/01/2026
Microcontr.	XIAO ESP32-S3 Sense (vision + inférence FOMO)	Seeed Studio	1	11,99 €	11,99 €	01/02/2026
	XIAO MG24 Sense (remplacé par ESP32-C3)	Seeed Studio	1	9,99 €	9,99 €	01/02/2026
	Livraison DHL + frais PayPal (Seeed)	Seeed Studio	—	—	6,27 €	01/02/2026

TABLE 4 – Récapitulatif des dépenses réelles du projet R2B2 — budget alloué : 120 €

Catégorie	Désignation	Fournisseur	Qté	P.U.	Total	Date
Énergie	Accumulateurs Samsung INR18650-25R 2500 mAh	NKON	7	2,56 €	17,92 €	01/02/2026
	Livraison (NKON)	Asendia	—	—	4,63 €	01/02/2026
	BMS 7S 30 A (protection Li-ion)	AliExpress	1	5,19 €	5,19 €	29/01/2026
	Chargeur 29,4 V 2 A (pack 7S)	AliExpress	1	5,69 €	5,69 €	29/01/2026
	Carte abaisseur de tension XL4015	AliExpress	1	1,39 €	1,39 €	29/01/2026
	Supports batterie 18650 (×3 et ×4)	AliExpress	2	—	3,58 €	25/03/2026
Électronique	Driver pont en H BTS7960 (×2)	AliExpress	2	3,11 €	5,25 €	29/01/2026
	Interrupteur 15×20 mm (×5)	KCD1 AliExpress	1 lot	1,75 €	1,48 €	29/01/2026
	Connecteurs 5,5×2,5 mm (×10) + capuchons	DC-022B AliExpress	1 lot	—	0,13 €	09/02/2026
Mécanique / 3D	Courroie GT2 6 mm – 188 mm (×2)	AliExpress	2	1,33 €	0,13 €	09/02/2026
	Courroie GT2 6 mm – 400 mm (×3)	AliExpress	3	1,39 €	4,17 €	09/02/2026
	Filament PETG 1 kg (pièces imprimées)	Alibaba	1	5,88 €	5,88 €	04/03/2026
	Visserie vrac (boulons, vis, écrous)	Leroy Merlin	—	—	0,37 €	12/02/2026
	Boulons M4×40 (×15) + visserie vrac	Leroy Merlin	—	—	5,23 €	13/03/2026
TOTAL réel dépensé					134,45 €	

TABLE 5 – Récapitulatif des dépenses réelles du projet R2B2 — budget alloué : 120 €

11. Démonstration

11.1 Scénario de démonstration

La démonstration finale de R2B2 a été organisée autour de cinq sous-systèmes fonctionnels, présentés de manière indépendante afin de mettre en valeur chacune des chaînes techniques développées au cours du projet.

11.1.1 Mesure de vitesse par capteur à effet Hall et transmission UART

Le premier sous-système présenté illustre la chaîne complète de mesure de vitesse. Le capteur SS49E, positionné en regard de la couronne magnétique fixée sur l'arbre moteur, délivre un signal sinusoïdal dont la fréquence est proportionnelle à la vitesse de rotation. Ce signal est directement échantillonné par l'ADC 12 bits du FPGA DE10-Lite ; le module `adc_period` en extrait la période en temps réel par isolation des demi-cycles successifs. La valeur calculée est encapsulée dans une trame UART et transmise au XIAO ESP32-C3, qui la relaie vers un ordinateur via la liaison série USB. Un script Python lit en continu les trames reçues sur le port série et affiche en direct la courbe du signal ADC ainsi que la vitesse de rotation correspondante en tr/min. Lors de la démonstration, la variation de la tension d'alimentation du moteur produit une évolution visible et immédiate de la courbe et de la valeur de vitesse affichée, validant l'ensemble de la chaîne d'acquisition, de traitement FPGA et de supervision PC.

11.1.2 Système de préhension — ouverture et fermeture de la pince

Le deuxième point de démonstration concerne le module de ramassage. La pince, actionnée par le moteur pas à pas 28BYJ-48 piloté en demi-pas depuis le FPGA, effectue une séquence complète d'ouverture et de fermeture sur une balle de tennis réelle. La saisie est franche et reproductible, validant à la fois la conception mécanique de la pince imprimée en PETG et le séquençage de la machine à états VHDL qui gère les huit phases du demi-pas ainsi que la protection thermique du moteur.

11.1.3 Détection d'obstacles par capteur à ultrasons

Un capteur HC-SR04 est présenté en fonctionnement autonome. Le module VHDL dédié pilote l'impulsion de déclenchement et mesure le temps de vol de l'écho avec une résolution à 50 MHz. La distance estimée est affichée en temps réel ; le passage de la main devant le capteur à moins de 200 mm déclenche le signal d'arrêt d'urgence, illustrant concrètement le mécanisme de sécurité de cohabitation homme-machine.

11.1.4 Interface de supervision graphique

Le monitor graphique Python est présenté en mode démonstration avec des données simulées. L'interface affiche en temps réel la vitesse des deux roues, l'angle de consigne, l'état du robot ainsi que les alertes ultrasons. Cette visualisation a été utilisée tout au long du développement pour diagnostiquer les dérives de l'asservissement et valider les seuils de détection sans modifier le firmware embarqué.

11.1.5 Détection de balle par apprentissage

Enfin, le module de vision est présenté en conditions réelles. Le XIAO ESP32-S3 Sense exécute le modèle FOMO entraîné sur Edge Impulse Studio à partir d'un jeu de données constitué spécifiquement pour ce projet. Pointée vers une balle de tennis posée au sol, la caméra détecte et localise la balle en continu à une période d'inférence de 1,2 s, en affichant l'angle de rotation calculé et le seuil de distance estimé. Le taux de détection observé dépasse 90 % en conditions d'éclairage intérieur standard, avec un taux de rejet des faux positifs supérieur à 95 %.

11.2 Difficultés rencontrées et solutions apportées

Au-delà des obstacles purement techniques, le projet a mis en évidence des difficultés d'ordre humain et organisationnel. L'**hétérogénéité de l'investissement** au sein du groupe — motivation et disponibilité variables selon les périodes de l'année — a fragilisé la continuité du développement. La **dispersion des compétences** a également constitué un frein : certaines tâches, comme la CAO et l'impression 3D, reposaient sur un seul membre du groupe, créant des goulots d'étranglement dès que plusieurs pièces devaient être conçues ou reproductibles en parallèle.

11.2.1 Circuit analogique SS49E — Signal inexploitable

Le signal issu du SS49E atteignait à peine $2,5\text{ V} \pm 1\text{ mV}_{pp}$ à 3 Hz, rendant la détection analogique des transitions magnétiques irréalisable malgré plusieurs semaines de débogage. La solution retenue consiste à transmettre le signal directement à la FPGA qui la quantifie via son ADC brut 12 bits et applique un traitement pour y extraire les périodes du signal

11.2.2 Roue folle — Axe recentré a posteriori

La version initiale plaçait l'axe de pivotement sous le centre de la roue, provoquant une résistance aux changements de direction. L'analyse de solutions commerciales a révélé le principe de l'axe déporté, conduisant à une reconception complète sous Fusion 360 et réimpression en PETG.

11.2.3 FSM UART — Données instables à l’envoi

Le strobe `i_send` et la mise à jour de `s_tx_data` survenaient dans le même cycle, corrompant les octets émis. La scission en deux états `LOAD_BYTE` / `SEND_BYTE` (données stables au cycle n , strobe au cycle $n + 1$) a résolu le problème, appliqué également dans `uart_mux.vhd`.

11.2.4 Pack batterie — Incident thermique et mesures correctives

Le pack batterie 7S1P a fait l’objet d’un incident lors de sa première phase d’assemblage. Les cellules étaient soudées directement entre elles, sans support mécanique, avec des languettes métalliques. Un faux contact a provoqué un court-circuit localisé : une odeur de brûlé est apparue, suivie de l’incandescence d’une des cellules. L’incident n’a heureusement causé ni blessure ni dommage matériel sur l’environnement de travail.

L’origine du problème réside dans la capacité de décharge élevée des batteries — jusqu’à 20 A en continu — qui rend tout faux contact immédiatement destructeur, sans temps de réaction possible. Trois mesures correctives ont été mises en place :

- Acquisition de **supports individuels pour cellules 18650**, maintenant chaque cellule mécaniquement et isolant les contacts latéraux ;
- Réalisation de toutes les **soudures à vide** (supports sans cellules insérées), les cellules n’étant introduites qu’une fois la totalité des connexions vérifiées ;
- Intégration du pack batterie dans le châssis de façon **amovible**, permettant une extraction rapide en cas d’incident et facilitant la maintenance ou le remplacement d’une cellule défectueuse.

Cet incident a sensibilisé l’ensemble du groupe aux risques spécifiques des batteries Li-ion haute décharge et à l’importance des précautions d’assemblage sur ce type de pack.

11.3 Travaux restants pour finaliser le robot

L’état actuel de R2B2 constitue une base technique solide, mais plusieurs modules critiques manquent pour obtenir un robot pleinement autonome et opérationnel. Les développements restants peuvent être regroupés en cinq axes.

11.3.1 Boucle de contrôle vitesse — Double PI synchronisé

Chaque roue motrice nécessite un régulateur PI indépendant, alimenté par la mesure de fréquence issue du capteur SS49E. Un module de contrôle supérieur doit synchroniser les deux PI pour garantir une trajectoire rectiligne et piloter les virages par différentiel de consigne. L’architecture FPGA est prête à accueillir ces modules ; seule leur implémentation en VHDL et leur calibration restent à réaliser.

11.3.2 Réception UART FPGA ← ESP32-C3

La chaîne UART est actuellement unidirectionnelle (FPGA → ESP32-C3). Un module `uart_rx` côté FPGA est nécessaire pour recevoir les consignes de vitesse et de direction émises par l'ESP32-C3, fermant ainsi la boucle de contrôle entre le microcontrôleur et le FPGA.

11.3.3 Algorithme de déplacement — ESP32-C3

L'intelligence de navigation doit résider dans l'ESP32-C3 : génération des consignes de trajectoire, gestion des séquences de collecte (approche, saisie, demi-tour), et arbitrage entre les différents sous-systèmes. Cet algorithme, central pour l'autonomie du robot, n'a pas été entamé.

11.3.4 Traitement des coordonnées balles — FPGA

L'ESP32-S3 Sense transmet les coordonnées des balles détectées via UART. Un module de traitement côté FPGA (ou ESP32-C3) doit parser ces trames, convertir les coordonnées pixel en angle et distance, et les fournir à l'algorithme de déplacement comme cibles de navigation.

11.3.5 Intégration des capteurs ultrasons

Les huit capteurs ultrasons sont câblés et testés individuellement, mais non intégrés dans la logique de décision. Il reste à implémenter leur gestion dans la FSM FPGA et dans l'algorithme ESP32-C3 pour assurer l'évitement d'obstacles en temps réel et alimenter l'occupancy map de supervision.

11.3.6 Lien Bluetooth et supervision

Le module `ble_robot` permettant de relier l'ESP32 au Tennis Robot Monitor n'a pas été finalisé. Son implémentation permettrait une supervision sans fil complète : télémétrie en direct, visualisation de la trajectoire et intervention à distance.

11.3.7 Campagne de tests et intégration système

Une fois ces modules développés individuellement, une phase d'intégration et de débogage système serait indispensable : tests de régression sur banc, validation des boucles fermées, calibration des régulateurs PI sur terrain, tests de robustesse mécanique et électrique. Ce type de projet, impliquant autant de couches logicielles, matérielles et mécaniques interdépendantes, **requiert réalistement une année supplémentaire de travail** pour atteindre un niveau de fiabilité démontrable en conditions réelles.

12. Conclusion

R2B2 est un projet de robotique autonome couvrant un spectre technique important pour un projet de cursus : électronique analogique, mécanique imprimée 3D, programmation FPGA en VHDL, vision embarquée par deep learning et supervision logicielle temps réel. En un an, nous avons conçu et réalisé un nombre conséquent de modules — mécaniques, électroniques et logiciels — dont plusieurs sont démontrables et fonctionnels. Pour autant, l'intégration de l'ensemble de ces briques n'a pas pu être finalisée : R2B2 n'est pas en état de se déplacer de façon autonome et d'opérer seul sur un terrain de tennis.

Ce projet nous a permis de confronter les modèles théoriques à la réalité du prototypage : signaux inexploitable, débogage non concluant, mécaniques à repenser, incident sur pack batterie. Nous retenons ainsi de cette année qu'un projet d'ingénierie se construit autant sur ses défaillances que sur ses succès.

Les évolutions possibles de R2B2 sont nombreuses et constituent un périmètre de travail réaliste pour une promotion future. La finalisation de la boucle PI fermée, l'algorithme de navigation, l'intégration des capteurs ultrasons dans la logique de décision et le lien Bluetooth représentent à eux seuls une année de développement à part entière. À plus long terme, une infrastructure **UWB** sur les poteaux du filet et un **correcteur LQR** sur FPGA transformeraient R2B2 en une plateforme robotique aboutie, digne d'un usage en conditions réelles.

Nous espérons que ce travail servira de base solide à une équipe future qui pourra s'appuyer sur l'architecture existante pour franchir les étapes restantes. R2B2 n'est pas un projet terminé — c'est un projet qui mérite de l'être.

13. Bibliographie

- [1] WALL STREET JOURNAL, *The Numbers Behind the Wimbledon Serve*, 2012. adresse : <https://www.wsj.com/articles/SB10001424052702304724404577492903106636484>.
- [2] FÉDÉRATION FRANÇAISE DE TENNIS, *Prévention des blessures au tennis*, 2023. adresse : <https://www.fft.fr/pratique/sante-et-bien-etre>.
- [3] TENNIBOT INC., *Tennibot – Autonomous Tennis Ball Collector*, 2024. adresse : <https://www.tennibot.com/>.
- [4] SLINGER BAG INC., *Slinger Bag – Portable Tennis Ball Launcher*, 2024. adresse : <https://www.slingerbag.com/>.
- [5] INTEL / TERIC, *DE10-Lite User Manual*, 2023. adresse : <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=1021>.
- [6] SEEED STUDIO, *XIAO ESP32-C3 Wiki*, 2023. adresse : https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/.
- [7] SEEED STUDIO, *XIAO ESP32-S3 Sense Wiki*, 2023. adresse : https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/.
- [8] ELECFREAKS, *HC-SR04 Ultrasonic Sensor Datasheet*, 2022. adresse : <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.
- [9] HONEYWELL, *SS49E Linear Hall-Effect Sensor – Datasheet*, 2022. adresse : <https://sensing.honeywell.com/SS49E>.
- [10] ALLEGRO MICROSYSTEMS, *A3141, A3142, A3143, A3144 Hall-Effect Switches – Datasheet*, 2020. adresse : <https://www.elecrow.com/download/A3141-2-3-4-Datasheet.pdf>.
- [11] JGB MOTORS, *JGB37-520 DC Geared Motor – Datasheet*, Moteur 24 V 960 RPM, couple 3 kg·cm, encodeur effet Hall intégré, 2023. adresse : <https://www.aliexpress.com>.
- [12] MOUSER ELECTRONICS, *28BYJ-48 Stepper Motor – Datasheet*, 2020. adresse : <https://www.mouser.com/datasheet/2/758/stepd-01-data-sheet-1143075.pdf>.
- [13] TOWERPRO, *MG996R Metal Gear Servo – Datasheet*, Couple max. 11 kg·cm, tension 4,8–7,2 V, 2019. adresse : <http://archive.communico.co.za/Content/Catalog/Documents/D0263792703.pdf>.
- [14] TOWERPRO, *SG90 Micro Servo – Datasheet*, Micro servo 9 g, couple 1,8 kg·cm, 2020. adresse : http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf.
- [15] INFINEON TECHNOLOGIES, *BTS7960B High Current PN Half Bridge – Datasheet*, Pont en H 43 A, 5,5–27 V, 2020. adresse : <https://dfimg.dfrobot.com/enshop/image/data/DRI0018/BTS7960.pdf>.

- [16] XLSEMI, *XL4015 5A 180kHz 36 V Buck DC-DC Converter – Datasheet*, Abaisseur de tension réglable 1,25–32 V, 5 A, 2020. adresse : <https://fr.scribd.com/document/331748365/XL4015-datasheet-pdf>.
- [17] SAMSUNG SDI, *INR18650-25R 2500mAh High Drain Cell – Datasheet*, Capacité 2500 mAh, décharge max. 20 A en continu, 2014. adresse : <https://www.batteryspace.com/prod-specs/6408.pdf>.
- [18] MISUMI, *GT2 Timing Belt – Technical Specification*, Courroie pas 2 mm, largeur 6 mm, 2023. adresse : <https://us.misumi-ec.com/vona2/detail/110302652060/>.
- [19] EDGE IMPULSE INC., *FOMO – Faster Objects, More Objects*, Architecture de détection d’objets temps réel pour microcontrôleurs, 2022. adresse : <https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/object-detection/fomo>.
- [20] J. JONGBOOM et al., « FOMO : Real-Time Object Detection on Microcontrollers, » in *Proceedings of the tinyML Research Symposium*, 2022. adresse : https://www.edge-ai-vision.com/wp-content/uploads/2022/05/E3W04_Jongboom_Edge_Impulse_2022.pdf.
- [21] SEEED STUDIO, *SenseCraft AI – Model Assistant for Edge Devices*, 2024. adresse : <https://sensecraft.seeed.cc/ai/>.
- [22] TEXAS INSTRUMENTS, *Understanding UART – Application Report SLAA749A*, 2019. adresse : <https://www.ti.com/lit/an/slaa749a/slaa749a.pdf>.
- [23] IEEE, *IEEE Standard VHDL Language Reference Manual – IEEE Std 1076-2008*, 2009. adresse : <https://ieeexplore.ieee.org/document/4772740>.